ArkPrism: Evidence-Carrying Privacy API Resolution for ArkTS Taint Analysis

ANONYMOUS AUTHOR(S)

Static privacy analysis for OpenHarmony/ArkTS depends on a step that conventional source–sink configuration leaves implicit: resolving a platform API identity and, when the API produces modeled data, binding it to the program value that carries that data. Package migration, manager receivers, property reads, and IR aliases can erase identity evidence before analysis begins; member-name matching can instead confuse unrelated application code with a platform source.

We present ArkPrism, an evidence-preserving static analysis that resolves the triple ⟨identity, witness, carrier⟩ from package, namespace, receiver, access-kind, SDK, and IR evidence. Well-formed return, argument, and callback carriers seed an ArkTS-specific IFDS analysis over initializers, closures, exceptions, and access paths. A bounded continuation analysis uniquely recovers Promise-`then` gaps and independently corroborates selected callback and `await` paths. Provenance-preserving evidence graphs retain the identity witness, call context, sink, path producer, and source location while keeping privacy-data and framework-input paths distinct.

Within a fixed rule set and statically enumerable candidate universe, a source-first benchmark shows that ArkPrism recovers all 90 gold occurrences at exact locations and rejects all 96 candidate collisions. It classifies all 64 matched identity cases correctly. On the complete 67-case HapBench suite, ArkPrism achieves 100% case-level precision and specificity, 97.09% F1, and 98.04% endpoint-pair precision. A balanced asynchronous benchmark attributes nine Promise-`then` cases uniquely to continuation recovery. A structured source/IR audit confirms all 90 sampled real-project privacy-data paths and 114/120 paths overall. Across 1,014 projects, ArkPrism identifies 2,336 privacy-API usages and 243 privacy-data paths without resource fallback.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → *Software defect analysis.*

Additional Key Words and Phrases: OpenHarmony, ArkTS, static analysis, privacy APIs, taint analysis, call-chain analysis

ACM Reference Format:
Anonymous Author(s). 2027. ArkPrism: Evidence-Carrying Privacy API Resolution for ArkTS Taint Analysis. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027).* ACM, New York, NY, USA, 33 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

OpenHarmony applications use ArkTS, a TypeScript-derived language with declarative ArkUI components, framework-managed callbacks, and system APIs exposed through historical `@ohos.*` and newer `@kit.*` packages [28, 32, 41]. These features complicate a task that precedes every privacy-flow query: identifying which program value is produced by a configured platform API. The same member token may denote application code, a third-party library, or a platform operation; the platform operation may appear as an imported namespace call, a property read, an IR alias, or a call on a manager returned by a factory.

ArkAnalyzer provides reusable IR, control-flow, call-graph, and pointer-analysis facilities for ArkTS [14], and HapFlow builds an IFDS taint analysis on this foundation [15]. Their availability makes whole-program ArkTS analysis practical, but it also exposes a front-end boundary: a solver can propagate only values that have first been bound to source identities. HapFlow's namespace–method loader maps configured entries to SDK method signatures. Its configuration schema does not encode executable property sources or retain the package/factory witness needed to decide a member on a manager receiver. Once source identity or its carrier is lost, downstream propagation cannot reconstruct the missing initial fact.

Consider `helper.createAsset(...)` after `helper` is returned by `photoAccessHelper.getPhotoAccessHelper(ctx)`. The member `createAsset` is common application vocabulary; its receiver origin establishes the platform identity. Conversely, accepting every same-named call in a file importing a media package would introduce collisions. Compound APIs such as `request.agent.create` create a related problem after IR lowering, while `deviceInfo.brand` is an executable property rather than a call. A useful analysis must therefore answer three linked questions: *which API is this*, *which value carries modeled data, if any*, and *which evidence justifies both decisions*.

We present ArkPrism, an evidence-preserving static analysis for ArkTS privacy auditing. For each candidate, it computes $\langle identity, witness, carrier \rangle$ from package migration, namespace/import evidence, receiver type or origin, member, access kind, SDK declarations, and IR aliases. Generic members such as `on`, `start`, and `close` require receiver or target evidence rather than lexical namespace proximity. A carrier is present only for a well-formed taint-query rule: return values and callback parameters model privacy data, while explicit framework arguments remain a separate source kind. Executable properties are retained as detector identities and support detector-local tracing; the evaluated IFDS inventory does not seed property values. ArkTS initializer, closure, exception, and access-path recovery then propagates typed facts, while a bounded continuation analysis uniquely fills Promise-`then` gaps and independently corroborates selected callback and `await` paths.

The resulting provenance-preserving evidence graph retains the acceptance witness, optional source carrier, call context, detector-local sink observations, configured-query paths, permissions, and source locations. It also keeps three evidence universes explicit: a recognized API usage need not produce a path; detector-local sinks are not the IFDS sink set; and framework-provided inputs are not silently counted as privacy-API outputs. This separation makes every relation traceable to its producing analysis and prevents a broad lifecycle model from inflating the core privacy-data claim.

Our evaluation uses separate oracles for identity, flow classification, mechanism attribution, and scale. Within the fixed rule and statically enumerable candidate universe of a source-first real-project benchmark, ArkPrism recovers all 90 gold occurrences at exact locations and rejects all 96 candidate collisions. It classifies all 64 matched identity cases correctly. On the complete 67-case HapBench suite, ArkPrism achieves 100% case-level precision and specificity, 97.09% F1, and 98.04% endpoint-pair precision. ArkAsyncBench attributes nine Promise-`then` cases uniquely to continuation recovery. A source/IR audit of 120 stratified real-project paths confirms all 90 sampled privacy-data paths and 114/120 paths overall; the six rejected candidates are confined to the separately labeled framework-input layer. Across 1,014 projects, ArkPrism identifies 2,336 privacy-API usages and 243 privacy-data paths without resource fallback.

This paper makes three contributions:

- **Evidence-carrying API identity.** We formulate ArkTS privacy-API recognition over package migration, receiver evidence, access kind, and IR aliases, covering direct calls, manager/helper receivers, compound namespaces, and executable properties while retaining an explicit match witness.
- **Carrier- and provenance-aware propagation.** We bind well-formed query identities to typed source carriers, recover ArkTS-specific IR semantics inside IFDS, and compose it with bounded continuation recovery. A unified provenance-preserving evidence graph retains identity, context, sink, path, and source-location witnesses without conflating detector, privacy-data, and framework-input evidence.

- **A multi-level empirical evaluation.** We contribute source-first identity benchmarks, a complete HapBench comparison, a balanced asynchronous benchmark, a 120-path semantic audit, and a clone-aware 1,014-project study that jointly establish identity accuracy, path quality, and scalability.

2 Background and Motivation

2.1 OpenHarmony and ArkTS

OpenHarmony applications are organized around abilities, pages, ArkUI components, lifecycle methods, and event callbacks [27, 28, 41]. ArkTS preserves much of the TypeScript surface syntax, but introduces stricter typing and a declarative UI model. In ArkUI, a component's `build` method describes UI structure, while lifecycle callbacks such as `aboutToAppear` and event handlers such as `onClick` are implicitly invoked by the framework. These properties make ArkTS attractive for application development, but complicate static analysis because important control-flow edges are not always explicit in source code.

Unlike traditional Android applications, where many lifecycle and callback conventions have been studied for years, OpenHarmony combines a relatively new application model with a rapidly evolving SDK. An ArkTS page may contain ordinary methods, lifecycle methods, anonymous functions, class-field callbacks, Promise handlers, and declarative UI builder code in the same file. The compiler and analyzer may lower these constructs into generated methods such as `%AM`-style anonymous methods, `%dflt`, `%statInit`, and `%instInit`. These generated methods are not noise for privacy analysis: they often contain callback assignments, page initialization, or API wrapper setup. An analysis that filters them out too aggressively can lose real paths.

OpenHarmony also exposes a broad API ecosystem for device identity, account information, clipboard access, sensors, camera, media, location, network state, Wi-Fi, Bluetooth, telephony, calendar, and other capabilities. Some APIs have migrated from historical `@ohos.*` packages to newer `@kit.*` packages, while many apps still contain mixed imports, aliases, default imports, and manager-style APIs. For example, a project may import device information from `@ohos.deviceInfo`, from `@kit.BasicServicesKit`, or through a default alias. Similarly, APIs such as pasteboard and request-agent operations may be invoked directly from a namespace or indirectly through an object returned by a manager factory.

This evolution creates a tension: exact package matching separates historical and consolidated exports of the same API, while method-name-only matching accepts common verbs without an ownership witness. A resolver must normalize documented migrations at namespace granularity while preserving receiver and member constraints.

2.2 Foundational Static Analysis for ArkTS

ArkAnalyzer provides ArkTS-aware code representation, IR transformation, Scene construction, call-graph construction, and basic data-flow support [14]. HapFlow builds on ArkAnalyzer to provide inter-procedural taint analysis for OpenHarmony, addressing lifecycle/callback entry modeling, closure-captured variables, and source/sink extraction [15].

ArkAnalyzer and HapFlow play different roles: ArkAnalyzer is a representation and graph-construction layer (Ark files, classes, methods, CFG, imports, call graphs); HapFlow is a taint-analysis layer (whether configured sources reach configured sinks). ArkPrism is built between and around these layers, asking a different question: can we recover the evidence that an auditor needs to understand a privacy-sensitive behavior?

Privacy auditing needs a *behavior-level* explanation: what API is used, how it is reached, what exposure points are downstream, what category and permission are involved, and how patterns

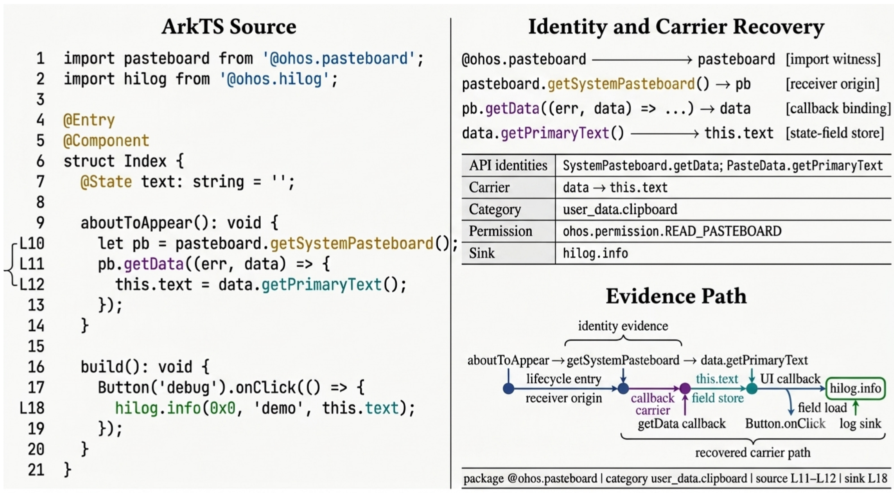

Fig. 1. Motivating ArkTS example and the evidence recovered by ArkPrism.

appear across a corpus. This distinction affects evaluation: a taint benchmark tests crafted source-to-sink cases, while a corpus audit needs API recognition against source evidence, call-chain coverage, sink distribution, and manual confirmation.

2.3 Provenance-Preserving Evidence Graphs

For each recognized privacy-sensitive API usage u, ArkPrism constructs a provenance-preserving evidence graph G_u that connects four layers: the API usage itself, the calling context that reaches it, downstream exposure points (sinks), and semantic metadata (package, permission, category). The formal definition is in Section 3. This structure deliberately separates several levels: a privacy-sensitive API usage may be real even when no downstream sink is found; a call chain may be locally precise even when no trusted entry can be recovered; a taint path may indicate data reachability but not necessarily a policy violation.

2.4 Motivating Example

Figure 1 shows a simplified ArkTS pattern. The app obtains the system pasteboard, reads data in a callback, stores the text in a component state field, and later logs it from a UI interaction. A source-to-sink analyzer should report the data flow. A provenance-preserving evidence graph should additionally explain that the source belongs to the clipboard category, that it is reached from a component lifecycle or callback context, that the sink is log output, and that the source is associated with a package/namespace import.

This example illustrates why a single analysis layer is insufficient: recognition must handle the indirect receiver pb, call-chain recovery must include lifecycle and UI callback edges, taint analysis must handle callback-provided source data, and sink analysis must classify `hilog.info` as log exposure. ArkPrism combines these layers in one report.

2.5 Analysis Requirements

The ArkTS setting induces six recurring analysis requirements: robust project-layout discovery, namespace-constrained package migration, compound-namespace recovery after IR lowering, manager-receiver provenance, callback-edge recovery, and typed propagation of callback-carried values. Section 3 maps these requirements to explicit analysis stages, while Section 5 tests them with identity, flow, and asynchronous oracles.

3 Design of ArkPrism

ArkPrism resolves the *source identity* bottleneck in ArkTS privacy analysis and produces provenance-preserving evidence graphs. ArkAnalyzer provides the ArkTS IR, project Scene, and call-graph substrate [14]; HapFlow provides IFDS-based taint propagation over OpenHarmony programs [15]. On this substrate, ArkPrism adds an evidence-carrying domain layer: package- and receiver-aware API identity resolution, bounded asynchronous carrier/context recovery, and provenance-preserving graph assembly.

3.1 Problem Definition and Notation

Let a project be $P = \langle F, M, I, E_c \rangle$, where F is the set of ArkTS source files, M is the set of methods in ArkAnalyzer IR, I is the set of import bindings, and $E_c \subseteq M \times M$ is the initial call graph. ArkPrism deliberately separates detector rules R_D from IFDS query rules R_T. A detector rule is

$$r_D = \langle pkg, ns, mem, mode, factory, perm, cat \rangle, \tag{1}$$

where pkg and ns identify the declaring package and namespace, mem is a method or property token, $mode \in \{namespace, receiver, property\}$ distinguishes the access forms, $factory$ is an optional set of receiver-producing methods, $perm$ is permission evidence, and cat is a privacy category. The serialized rule inventory normalizes its legacy tri-state access field to this explicit mode. An IFDS query rule is

$$r_T = \langle sig, meta, carrier, i_{cb}, i_{src}, kind \rangle, \tag{2}$$

where sig is an SDK method signature; $meta = \langle module, ns, class, mem, params, lit \rangle$ retains the source identity needed when the frontend cannot resolve that signature; and $carrier \in \{return, arg, callback\}$ identifies the program value to seed. The source kind distinguishes typed privacy data from explicit framework inputs:

$$kind \in \{privacy\_data, framework\_input\}. \tag{3}$$

The optional lit component records a string-dispatched event name. For callback sources, i_{cb} is the callback-argument position and i_{src} is the sensitive callback-parameter position. A privacy-data return requires a non-`void` return type; a callback source requires a typed `Callback<T>` parameter and a valid carrier index; an argument source is accepted only as an explicitly modeled framework input. Rules without this carrier contract are rejected at load time. Keeping R_D and R_T explicit is essential: a permission-bearing API operation may belong to R_D without producing privacy data, and is therefore not automatically promoted into R_T.

API identity resolution. The first task is to compute a semantic identity for each candidate API usage. We define:

$$Id(v) = \langle Pkg,\ Ns,\ Recv,\ Mem,\ Acc \rangle, \tag{4}$$

where Pkg and Ns are resolved through a namespace-constrained migration map, $Recv$ traces the receiver object back to its source via bounded backward data-flow (e.g., `cameraMgr` ← `getCameraMgr(ctx)`), Mem is the method or property name, and $Acc \in \{direct, assign, indirect, property\}$ classifies the access pattern. Resolution also returns a witness

$$w \in \{Namespace, IRAlias, ExactType, CallTarget, FactoryOrigin, PropertyRead\}. \tag{5}$$

A usage u is recognized when:

$$\exists s \in Stmt(P),\ \exists r_D \in R_D :\ Accept(s, r_D, I, A, w), \tag{6}$$

where A is the alias relation recovered from imports, assignments, and IR-lowered namespace-member expressions. The recognized usage set is $U_D = \{u \mid \exists s, r_D, w : Accept(s, r_D, I, A, w)\}$.

Source carriers and seeds. For a statement s matching r_T, the initial taint fact is determined by the carrier rather than by the API name alone:

$$Seed(s, r_T) = \begin{cases} lhs(s), & carrier = return, \\ arg_{i_{src}}(s), & carrier = arg, \\ param_{i_{src}}(ResolveCb(arg_{i_{cb}}(s))), & carrier = callback. \end{cases} \tag{7}$$

Property occurrences in U_D provide right-hand-side values to detector-local tracing. The evaluated R_T inventory contains no property carrier, so these occurrences do not seed IFDS. Framework-provided `Want` values are query sources of kind *framework_input*, not privacy-API detector occurrences. These distinctions prevent detector coverage, privacy-data seeding, and framework-input analysis from being silently equated.

The resolution unit preserved downstream is

$$ResolveUsage(s) = \langle Id(s), w, c \rangle, \tag{8}$$

where $c = Seed(s, r_T)$ for an accepted, well-formed query rule and $c = \perp$ for detector-only evidence. This optional-carrier triple makes API recognition explicit without manufacturing a taint seed for operations whose sensitive value is not modeled.

Asynchronous carrier recovery. The second task is to bind a configured asynchronous source to its consumer. We define:

$$SrcBind(call, cb, param), \tag{9}$$

where *call* is the source API invoke statement, *cb* is the resolved callback method or Promise handler, and *param* is the specific parameter through which sensitive data flows. The implementation distinguishes native IFDS propagation from bounded post-IFDS carrier recovery and from callback edges used only for call-chain context; provenance records that distinction.

Provenance-preserving evidence graph. The third task is to construct, for each $u \in U_D$, an evidence graph:

$$G_u = \langle V_u, E_u, \lambda_u, \omega_u \rangle, \tag{10}$$

where V_u contains API, caller, sink, and taint-fact nodes; E_u contains call, detector-local sink, configured-query path, and endpoint-link edges. The labeling function λ_u records the semantic edge role. The origin function ω_u is reconstructed from the producing record:

$$\omega_u(e) = \begin{cases} callType(e), & e \in E_{call}, \\ detector\_local, & e \in E_{sink}, \\ \tau(q), & e \in E_{path}(q), \\ linkEvidence(e), & e \in E_{link}, \end{cases} \tag{11}$$

where call types distinguish direct, callback, and lifecycle-implicit context; $\tau(q)$ distinguishes IFDS, asynchronous-supplement, and corroborated paths; and link evidence distinguishes exact-statement from normalized-source-location joins. This representation preserves how each subgraph relation was produced without treating derivation type as a probability of correctness.

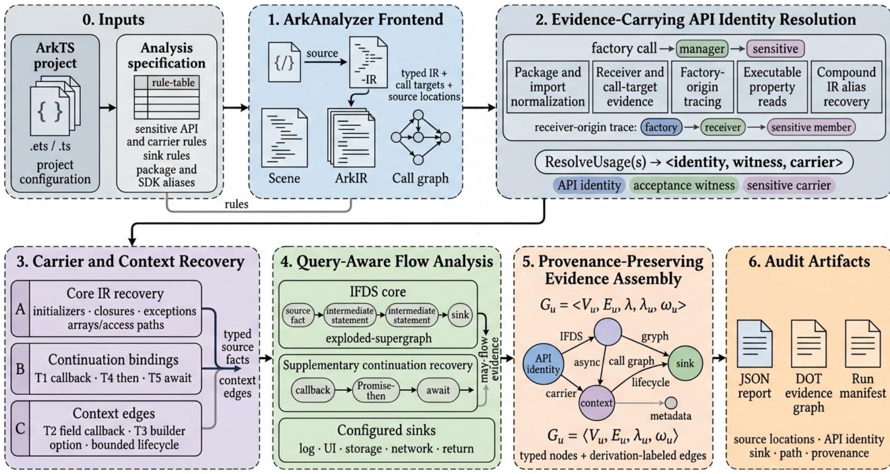

Fig. 2. ArkPrism analysis pipeline and evidence assembly.

3.2 Architecture Overview

Figure 2 gives the end-to-end architecture. ArkAnalyzer transforms each project into a Scene, ArkIR, and call graph; the analysis specification configures source identity, carrier, package-alias, and sink rules. ArkPrism then performs three connected analyses: (1) API identity resolution computing $\mathit{ResolveUsage}(s)$, (2) carrier propagation across IR and selected asynchronous continuations, and (3) evidence-graph assembly that preserves the derivation label $\omega(e)$ on every edge. Lifecycle ordering and call-graph context supply reachability evidence to this pipeline.

3.3 Running Example: Why Direct Calls Are Insufficient

Figure 3 shows an anonymized pattern distilled from manually reviewed benchmark projects. The example is intentionally small, but each highlighted source position corresponds to a real source-level evidence class seen during manual annotation: a manager object returned by a Harmony API, a member call on that manager, a compound namespace API that may be lowered through a temporary, and a local sink that helps an auditor inspect the behavior.

A direct-call baseline that only matches `namespace.method(...)` statements can recover neither `this.calendarMgr.getCalendar(...)` nor `agent.create(...)` reliably. The former is a member call whose receiver obtains its privacy meaning from `calendarManager.getCalendarManager`; the latter is a separate compound-namespace example whose `request.agent` prefix may be assigned to an IR temporary before `create` is invoked. ArkPrism treats both cases as rule- and evidence-matching problems over imports, aliases, receiver origins, and IR-lowered expressions, then retains lifecycle context, local sink evidence, configured-query paths, and source metadata instead of returning flat method-name hits.

3.4 Namespace-constrained Package Migration

OpenHarmony API evolution makes exact package matching brittle, but consolidated kit packages are not globally equivalent to their historical modules. We therefore define a directional migration relation over package–namespace pairs, $\mathcal{M} \subseteq (Pkg \times Ns) \times (Pkg \times Ns)$. For example, the `deviceInfo` export maps from `@ohos.deviceInfo` to `@kit.BasicServicesKit`, while unrelated exports in the same kit do not become compatible. A detector rule r_D and import binding i are compatible when:

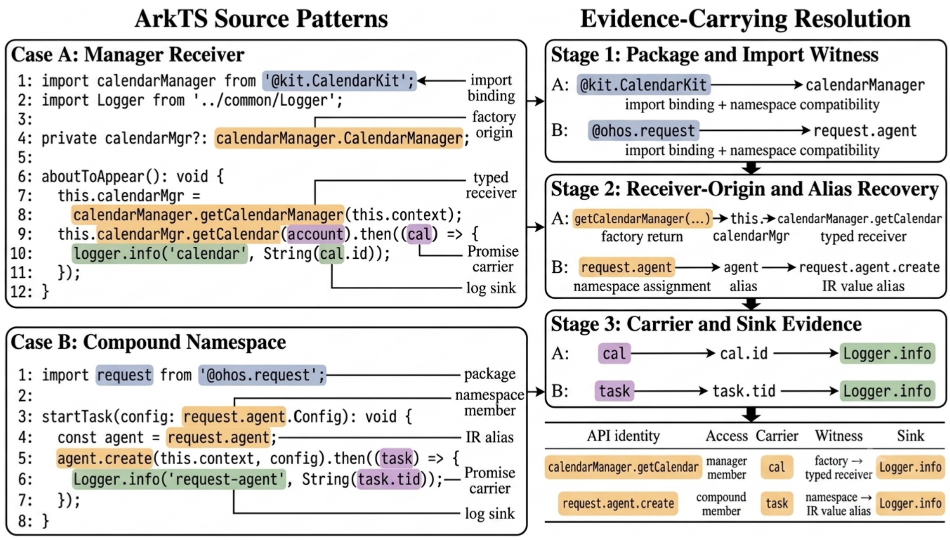

Fig. 3. Evidence-carrying resolution for manager receivers and compound namespaces.

$$Compat(i, r_D) \triangleq (i.pkg, i.ns) = (r_D.pkg, r_D.ns) \vee ((i.pkg, i.ns), (r_D.pkg, r_D.ns)) \in \mathcal{M}. \tag{12}$$

The analyzer also derives namespace aliases from import declarations. If a source file contains `import wifi from '@ohos.wifiManager'`, then $(wifi, \texttt{wifiManager}) \in A_{import}$. If it imports a kit package with a default binding, ArkPrism records both the textual alias and the canonical namespace. Matching therefore uses a normalized API key:

$$key(u) = \langle \mu(pkg_u, ns_u), \sigma(mem_u) \rangle, \tag{13}$$

where μ applies the migration/export map and import-alias normalization to a package–namespace pair, and σ normalizes method/property spelling. The report retains both $key(u)$ and the raw package signature so that normalized and SDK-specific identities remain auditable.

3.5 Package- and Receiver-aware API Identity Resolution

HapFlow's namespace–method loader resolves configured entries to SDK declarations, but its schema does not encode executable property reads or retain factory-origin evidence for manager/helper receivers. ArkPrism represents these access forms by computing a semantic identity and an acceptance witness for each candidate usage.

3.5.1 Identity Function. Given a candidate statement s, ArkPrism computes $Id(s)$ as defined in Section 3.1. The key components are $Recv(s)$ (receiver-origin tracing, detailed below) and $Acc(s)$ (access-kind classification). The rule mode denotes

$$Mode(r_D) = \begin{cases} \{direct, assign\}, & r_D.mode = namespace, \\ \{indirect\}, & r_D.mode = receiver, \\ \{property\}, & r_D.mode = property. \end{cases} \tag{14}$$

The candidate predicate is:

$$\begin{aligned} Match(s, r_D, I, A) = Compat(I_s, r_D) \\ \wedge\ Id(s).Mem{=}r_D.mem \\ \wedge\ Acc(s) \in Mode(r_D). \end{aligned} \tag{15}$$

Package compatibility only generates candidates. Acceptance additionally requires access-specific evidence:

$$\begin{aligned} Accept(s, r_D, I, A, w) \triangleq Match(s, r_D, I, A) \\ \wedge\ Sufficient(Acc(s), w, r_D), \end{aligned} \tag{16}$$

$$\mathcal{W}_{ind} = \{ExactType, CallTarget, FactoryOrigin, Namespace\}. \tag{17}$$

$$Sufficient(a, w, r_D) = \begin{cases} w \in \{Namespace, IRAlias\}, & a \in \{direct, assign\}, \\ w = PropertyRead, & a = property, \\ w \in \mathcal{W}_{ind}, & a = indirect. \end{cases} \tag{18}$$

For indirect calls, *Namespace* means an exact namespace receiver, not merely package presence. It is disabled for generic members such as `get`, `on`, `start`, `request`, and `close`; these require type, target, or factory-origin evidence. Property acceptance requires both $r_D.mode = property$ and an executable IR field read. Thus, package import or final-token agreement alone is never sufficient.

IFDS source-rule acceptance. The IFDS query uses SDK method signatures whenever ArkAnalyzer resolves a call target. For a call whose signature remains unknown after frontend recovery, matching falls back to the configured rule metadata rather than to method name alone. Let $Meta(r_T)$ retain the rule's module, namespace, declaring class, member, arity, parameter types, and any string-dispatched event literal. We define:

$$Exact_T(s, r_T) \triangleq TargetSig(s) = Sig(r_T), \tag{19}$$

$$\begin{aligned} Fuzzy_T(s, r_T) \triangleq UnkSig(s) \wedge Mem(s) = r_T.mem \\ \wedge\ ArityCompat(s, r_T) \wedge ParamCompat(s, r_T). \end{aligned} \tag{20}$$

The guard for a generic member and the guard for a string-dispatched overload are:

$$\begin{aligned} IdentityGuard(s, r_T) \triangleq \neg Generic(r_T.mem) \\ \vee\ ReceiverOrModuleWitness(s, Meta(r_T)), \end{aligned} \tag{21}$$

$$\begin{aligned} LiteralGuard(s, r_T) \triangleq \neg HasEventLiteral(r_T) \\ \vee\ ArgLiteral(s, 0) = EventLiteral(r_T). \end{aligned} \tag{22}$$

The source call is accepted iff

$$Accept_T(s, r_T) \triangleq Exact_T(s, r_T) \vee \big(Fuzzy_T(s, r_T) \wedge IdentityGuard(s, r_T) \wedge LiteralGuard(s, r_T)\big). \tag{23}$$

Here *ArityCompat* requires exact arity whenever the rule declares a parameter array, including an empty one. *ParamCompat* requires parameter names and types to match after normalizing imported type qualifiers, whitespace, array notation, callback aliases, and union order; rules with non-concrete return carriers are rejected before matching. Consequently, an unresolved call such as `avPlayer.on('error',...)` cannot match a configured geolocation or sensor event merely because both use on with the same arity. This predicate governs IFDS source seeding and is separate from the detector acceptance predicate above.

Table 1. API identity resolution examples.

Source pattern	Pkg. family	Ns./recv.	Origin	Member	Acc.	Acceptance witness
`id.getOAID(cb)`	BasicSvc	`id`	⊥	`getOAID`	direct	Import + namespace
`net=getDefaultNet()`	NetworkKit	`net`	⊥	`getDefaultNet`	assign	Import + assignment
`mgr.getData(cb)`	BasicSvc	`mgr`	`getMgr`	`getData`	indirect	Factory return
`deviceInfo.brand`	BasicSvc	`devInfo`	⊥	`brand`	property	Executable read

3.5.2 Receiver-Origin Tracing. The key addition beyond namespace–method loading is $Recv(s)$. For a statement $s : x.m(\ldots)$ where x is a local or field variable, ArkPrism traces x backward through assignments and instance-invoke bases to find its origin, subject to a tracing bound of k_{recv} steps (default 8) to avoid unbounded backward exploration:

$$Recv(s) = \begin{cases} Ns(def), & x \leftarrow invoke(q.f()),\ f \in factory(r_D), \\ Recv(def), & x \leftarrow y,\ y \text{ has known origin}, \\ TF(x), & type(x) = T,\ T \in SDK, \\ \bot, & \text{unresolvable in } k_{recv} \text{ steps}. \end{cases} \tag{24}$$

The first case covers the common pattern $mgr \leftarrow getManager(ctx);\ mgr.getData(cb)$, where the receiver mgr originates from a privacy API factory call. The second case propagates origin through intra-procedural assignment chains and instance-invoke bases. The third case (*TF*) uses an exactly compatible declared SDK type. Inter-method field assignments are not claimed as origin-proven unless the defining statement or an exact type/target identity is recovered; otherwise the call is rejected. This rule avoids silently treating package import plus method-name agreement as a typed call target.

3.5.3 Access-Kind Classification. Each recognized usage receives an *Acc* label:

- *direct*: bare invoke statement, e.g., `id.getOAID(cb)`;
- *assign*: invoke with return value assigned, e.g., `net = getDefaultNet()`;
- *indirect*: invoke on a traced receiver, e.g., `mgr.getData(cb)` where $Recv(mgr) \neq \bot$;
- *property*: field/property read, e.g., `deviceInfo.productModel`.

Table 1 illustrates how *Id* resolves each access kind.

IR-lowered multi-segment APIs require an additional alias relation A_{ir}. For assignments of the form $t := q.member$, ArkPrism records $(t, q.member) \in A_{ir}$. A later statement `t.create(...)` is matched as `q.member.create(...)` if the composed signature appears in R_D. This is essential for APIs such as `request.agent.create`, whose namespace can disappear after lowering.

3.6 Lifecycle State Machine Model

OpenHarmony applications follow a structured lifecycle managed by the system runtime. A `UIAbility` transitions through creation, foreground, background, and destruction phases; custom components follow appearance, build, page visibility, and disappearance phases; UI callbacks are registered during component build and triggered by user interaction. These transitions are *implicit*—the runtime invokes lifecycle methods without visible call edges in application code—so a static analyzer that treats each lifecycle method as an independent entry point misses cross-phase data flow.

ArkPrism models the OpenHarmony application lifecycle as a three-layer state machine. Let the lifecycle model be:

$$\mathcal{L} = \langle \Phi_a, \Phi_c, \delta_a, \delta_c, \delta_{cross} \rangle, \tag{25}$$

Table 2. Asynchronous carrier and context recovery rules.

Rule	Construct	Recovery relation	Required witness	Output	Domain
T1	Callback source	$Resolve(a_i) \to cb.param_j$	Typed or definition-backed callback	May-path	Post-IFDS
T2	Field callback	$Read(this.f) \to cb$	Traced field assignment	Call edge	Report CG
T3	Builder option	$Arg(bld, f) \to cb$	Named option field	Call edge	Report CG
T4	Promise then	$ret(src) \to then_j.param$	Source-owned then chain	May-path	Post-IFDS
T5	`async`/`await`	$resolve_{arg} \to await_{res}$	Definition-backed Promise	May-path	Post-IFDS

where Φ_a is the set of ability phases (e.g., `CREATING`, `FOREGROUNDED`), Φ_c the component phases (e.g., `APPEARING`, `BUILT`), $\delta_a \subseteq \Phi_a \times \Phi_a$ the ability transition relation, $\delta_c \subseteq \Phi_c \times \Phi_c$ the component transition relation, and $\delta_{cross} \subseteq \Phi_a \times \Phi_c$ the cross-layer transition relation. Cross-layer transitions capture implicit calls such as `onForeground` $\longrightarrow$ `aboutToAppear`.

The lifecycle modeler discovers ability classes by checking superclass inheritance from `UIAbility`, `Ability`, and extension ability base classes; component classes by checking superclass (`CustomComponent`, `ViewPU`), decorators (`@Component`), or the presence of two or more component lifecycle methods; and callback methods by scanning IR statements for `CommonMethod.*` invocations and extracting `FunctionType` arguments.

Lifecycle information is consumed in two distinct ways. For call-chain recovery, each transition $(m_i, m_j) \in \delta_a \cup \delta_c \cup \delta_{cross}$ becomes a provenance-labeled implicit edge. For IFDS, the model orders entry methods in a synthetic `DummyMain`: ability creation and foreground callbacks precede component initialization and UI callbacks, followed by background and destruction phases. The bounded default removes the standard back edge from the final entry to the loop header, preventing arbitrary destruction-to-creation cycles. Repeatable event classes may receive one additional explicit firing, but no unbounded loop. The IFDS solver derives interprocedural targets from the Scene and pointer information; it does not consume the separately augmented report call graph.

3.7 Asynchronous Carrier and Context Recovery

OpenHarmony privacy APIs often deliver sensitive data through asynchronous callbacks and Promise chains rather than direct return values. For example, `geoLocationManager.getCurrentLocation(callback)` passes location data to a callback parameter, while `wifiManager.getLinkedInfo().then(callback)` delivers Wi-Fi state through a Promise resolution. Standard call-to-return propagation does not by itself establish the connection between a source invocation and a callback parameter or Promise resolution, so these constructs require explicit transfer rules.

ArkPrism formalizes carrier binding as $SrcBind(call, cb, param)$ (Section 3.1). The five recovery rules do not all modify the same analysis. T1, T4, and T5 add bounded source-to-sink outcomes after the core IFDS run when ArkIR lacks an explicit continuation edge; T2 and T3 add callback edges to the report call graph and therefore improve context recovery rather than seeding IFDS facts. Table 2 makes this execution domain explicit.

3.7.1 T1: Callback Parameter Binding. When an R_T source accepts a callback as a direct argument at position i, the binding is $SrcBind(s, cb, param_j)$ where $cb = Resolve(a_i)$ and $param_j$ is the callback parameter selected by the rule's carrier metadata. The post-IFDS phase accepts only typed or definition-backed callbacks. Resolution applies three strategies in order:

$$Resolve(a_i) = \begin{cases} m_f, & a_i \in FuncType \wedge m_f = Lkp(a_i.sig) \\ m_c, & a_i \in ClosureType \wedge m_c = Lkp(Ext(a_i)) \\ m_l, & a_i \in Local \wedge m_l = Trc(a_i) \end{cases} \tag{26}$$

FunctionType resolution (*Lkp*) extracts the method signature from the compiler-generated function type and looks up the corresponding method in the declaring class. *ClosureType* resolution (*Ext+Lkp*) parses the closure variable name from the closure type string (e.g., `closures: __lambda1`) and performs a method name lookup. *Local* variable tracing (*Trc*) follows the definition chain: if a_i is assigned from a `new LambdaClass()` expression, the ClosureType of the constructor call provides the callback method. Example: `getData((err,data)⇒...)` resolves the arrow function to its IR method and binds `data` as the sensitive parameter.

The tracer seeds only $param_j$ selected by $r_T.i_{src}$, not every callback parameter. It propagates through ArkIR value-use edges and assignments, and accepts a sink only when a sink argument structurally depends on the current fact. IR-local names are never compared by substring, and control dependence alone does not constitute an explicit data-flow edge.

3.7.2 T2: Field-Stored Callback Binding. ArkUI components frequently store callback functions in class fields (e.g., `this.handler = () => {...}`). The callback is registered with a framework API at a later point, creating a temporal gap between definition and invocation. ArkPrism reads the field value at the registration site and traces it to the assignment, adding $Read(this.f) \longrightarrow cb$ to the report call graph. This handles the pattern where `this.onSelect` is passed to `List.onSelect(this.onSelect)` in a component's build method. T2 supplies entry/call context; it does not by itself assert a sensitive carrier or inject an IFDS fact.

3.7.3 T3: Builder-Option Callback Binding. ArkUI builder APIs accept configuration objects whose fields are callback functions (e.g., `bindPopup({builder: popupBuilder})`). T3 extracts the callback from a named option field, $Arg(bld, f) \longrightarrow cb$, and adds the recovered owner–builder edge to the report call graph. The pattern is structurally distinct from T1 because the callback is nested inside a lowered object initializer rather than passed as a positional source callback. As with T2, the edge provides call context and is not an IFDS seed.

3.7.4 T4: Promise-then Chain Binding. Promise `.then()` chains can carry a source return value through Promise resolution to a callback parameter. When the core IFDS result lacks that continuation, the bounded supplement recognizes three sub-patterns:

Direct chaining: $sourceApi(args).then(cb)$ creates $SrcBind(sourceStmt, cb, param)$ with $Edge(sourceStmt, cb_{param})$.

Variable chaining: $x = sourceApi(args);\ x.then(cb)$ creates $Edge(sourceStmt, x_{assign}) \cup Edge(x_{assign}, cb_{param})$.

Sequential chaining: $x.then(cb_1).then(cb_2)$ creates $\bigcup_i Edge(cb_{i-1}, cb_i)$, propagating taint through the chain.

3.7.5 T5: Async/Await Bridge Binding. An asynchronous function that awaits a Promise-wrapped source API creates an implicit binding between the resolved value and the local variable:

$$data = await\ promiseMethod() \implies Edge(resolve_{arg}, await_{res}) \cup Edge(await_{res}, data_{uses}) \tag{27}$$

where $resolve_{arg}$ traces from the `resolve()` call inside the Promise executor to the awaited result. The supplement requires a definition-backed Promise owner and separately labels the resulting path; it does not claim that the core IFDS supergraph contains the implicit executor-to-`await` edge.

3.7.6 Algorithm. Algorithm 1 summarizes the post-IFDS portion (T1, T4, and T5). It does not modify IFDS flow functions or the exploded supergraph. Instead, it scans configured sources whose continuation edge is absent, resolves only typed callbacks or definition-backed Promise owners, performs bounded local/interprocedural tracing, and appends separately provenance-labeled outcomes. T2 and T3 are constructed by the callback-augmented report call graph in Section 3.7; they are intentionally absent from Algorithm 1.

Algorithm 1 Post-IFDS Asynchronous Carrier Recovery

Require: Program Scene P, IFDS source rules R_T, existing outcomes O, bounds B
Ensure: Augmented outcomes O'; each new outcome preserves $\epsilon(src)$ and has provenance AsyncSupplement

```
A ← ∅
for all application methods m ∈ P do
    for all invoke statements s ∈ m do
        src ← CallSource(s, R_T)
        if src ≠ ∅ ∧ src.carrier = callback then                          ▷ T1
            cb ← Resolve(s.args[src.cbIdx])
            if cb ≠ ∅ ∧ src.srcIdx < |cb.params| then
                p ← cb.params[src.srcIdx]
                A ← A ∪ TraceToSink(cb, p, Path(s), ϵ(src), B)
            end if
        end if
        if s is a then invocation on base b then                          ▷ T4
            if b is a configured source invocation then
                si ← b
            else
                si ← FindSrcDef(m, b)
            end if
            if si ≠ ∅ then
                cb ← Resolve(s.args[0])
                if cb ≠ ∅ then
                    A ← A ∪ TraceThenToSink(m, si, cb, ϵ(si), B)
                end if
            end if
        end if
    end for
    for all await sites a ∈ m with promise p do                           ▷ T5
        ow ← FindPromiseOwner(p, R_T)
        if ow ≠ ∅ then
            rf ← CollectResolveFacts(ow)
            if rf ≠ ∅ then
                A ← A ∪ TraceToSink(m, a.res, Path(rf) ∪ Path(a), ϵ(ow), B)
            end if
        end if
    end for
end for
O′ ← Deduplicate(O ∪ A)
return O′
```

3.8 Callback-augmented Call-chain Recovery

Given a recognized usage u inside method m_u, call-chain recovery searches a reverse graph:

$$E_r = (E_c)^{-1} \cup (E_{invoke})^{-1} \cup (E_{cb})^{-1} \cup (E_{init})^{-1}. \tag{28}$$

Here E_c is the ArkAnalyzer call graph, E_{invoke} contains IR-level call edges absent from the graph, E_{cb} contains callback edges recovered from FunctionType/ClosureType values, field callback assignments, and builder-option callbacks, and E_{init} links generated initialization methods to their owning class or component context. The recovered chain set is:

$$\begin{aligned} C_u = \{\, \pi = (m_0, \ldots, m_u) \mid m_0 \in \mathit{Entry} \cup \mathit{Init} \cup \mathsf{Unk}, \\ (m_i, m_{i+1}) \in E_r^{-1} \,\}. \end{aligned} \tag{29}$$

Each chain receives an entry-evidence class:

$$\kappa(\pi) = \begin{cases} \mathit{framework}, & m_0 \in \mathit{Entry} \\ \mathit{initialization}, & m_0 \in \mathit{Init} \\ \mathit{local}, & \pi = (m_u) \text{ and no predecessor is found} \\ \mathsf{unk}, & \text{otherwise.} \end{cases} \tag{30}$$

ArkPrism does not collapse these classes into a single coverage number. A local fallback chain is useful evidence that the API usage exists in a method, but it is not a recovered framework-entry path. This distinction is preserved in the report and in the evaluation tables.

3.9 Sink and Taint Attachment

For each detector occurrence $u \in U_D$, local tracing extracts statements $K_D(u)$ that may expose privacy-related data through logging, UI display, storage, network transfer, or data return. Sink resolution requires an exact configured method token and compatible receiver/namespace evidence; in particular, substring agreement such as `set` versus `setData` is rejected.

The IFDS query is configured independently by R_T and its sink set K_T. Let S_P and S_F be source statements whose rules have kind *privacy_data* and *framework_input*, respectively. The solver computes

$$T_I \subseteq S_T \times K_T \times \mathit{Path}, \tag{31}$$

where $S_T = S_P \uplus S_F$ is the disjoint typed-source universe resolved from R_T. Each tuple $\langle s, k, \rho \rangle$ means that the carrier seeded at s may reach configured sink k along path ρ. Each seeded fact carries an immutable source witness

$$\epsilon(s) = \langle \mathit{kind}, \mathit{pkg}, \mathit{ns}, \mathit{cls}, \mathit{mem}, \mathit{carrier}, \mathit{sig}, \mathit{rule}, \mathit{loc} \rangle, \tag{32}$$

where $\mathit{kind} \in \{\mathit{privacy\_data}, \mathit{framework\_input}\}$ and the remaining fields identify the resolved API, carrier position, SDK signature, rule provenance, and normalized source location. Normal, call, return, and exceptional transfers copy $\epsilon(s)$ unchanged; fact and edge identities include it. Consequently, values reaching the same statement from different configured sources remain distinct, and the report obtains its source endpoint from $\epsilon(s)$ rather than inferring it from the first statement in ρ. The bounded asynchronous rules preserve the same witness and produce a separate set

$$T_A \subseteq S_T \times K_T \times \mathit{Path}, \tag{33}$$

and the report exposes $T_Q = T_I \cup T_A$, stratified as

$$T_P = \{q \in T_Q \mid \mathit{src}(q) \in S_P\}, \qquad T_F = \{q \in T_Q \mid \mathit{src}(q) \in S_F\}. \tag{34}$$

For duplicate endpoint/path outcomes, path provenance is retained as

$$\tau(q) \in \{\mathit{ifds}, \mathit{async\_supplement}, \mathit{both}\}. \tag{35}$$

A partial linkage relation

$$L \subseteq U_D \times T_P \tag{36}$$

is created only when project, normalized source location, and compatible API identity agree. Framework-input paths are never linked to privacy-API detector occurrences. Linked privacy-data paths are attached to the corresponding G_u; unlinked configured-query paths remain project-level taint evidence rather than being forced onto a detector occurrence. Consequently, detector-local sink-positive projects and configured-query-path-positive projects are not set-inclusion counts. The corpus evaluation reports both evidence universes and both source kinds separately.

Construction invariants. Three invariants connect the formal contract to the implementation. First, *well-formed seeding* requires

$$q \in T_Q \Rightarrow \exists r_T, \epsilon : \mathit{WellFormed}(r_T) \wedge \mathit{SeededBy}(q, r_T, \epsilon), \tag{37}$$

so every emitted path has one valid carrier and source kind. Second, *witness preservation* requires $\epsilon_{out} = \epsilon_{in}$ across every normal, call, return, exceptional, and supplementary transfer; facts from different sources therefore remain distinct even when they reach the same statement. Third, *link safety* requires

$$\begin{aligned}(u, q) \in L \Rightarrow\ & \mathit{kind}(q) = \mathit{privacy\_data} \wedge \\ & \mathit{ProjectMatch}(u, q) \wedge \mathit{LocationMatch}(u, q) \wedge \mathit{IdentityCompatible}(u, q).\end{aligned} \tag{38}$$

The loader, fact-identity keys, transfer functions, and evidence linker enforce these properties by construction; the evaluation validates their observable consequences through source-rule resolution, endpoint integrity, and path audits.

3.9.1 Provenance Annotation. The report stores provenance at the level where it is produced. A call-chain relation carries a `callType`; a configured-query path carries $\tau(q) \in \{\mathit{ifds}, \mathit{async\_supplement}, \mathit{both}\}$; and an endpoint link records `exact_statement` or `source_location`. Detector-local sink edges are identified by their containing record. Evidence-graph assembly maps these fields to ω_u rather than replacing them with a synthetic global confidence score. Consequently, an asynchronous supplement cannot be presented as an IFDS path, and a normalized-location join remains distinguishable from an exact-statement join.

3.10 Analysis Contract

ArkPrism's recognition claim is parameterized by three explicit inputs: ArkAnalyzer IR for the source construct, the fixed API rule set, and the namespace, receiver, origin, or target-signature evidence required by *Accept*. The analysis targets statically represented ArkTS code; native code, dynamic loading, reflection-like runtime dispatch, and APIs absent from the rule set are outside the query domain. A recognized API establishes an executable use under this contract, while policy-level violation assessment remains a downstream audit decision.

IFDS and asynchronous-supplement results are may-flows under their respective source/sink and reachability models. The evidence model preserves their producer and admits a detector occurrence as an IFDS source only through a source-endpoint link. The solver distinguishes complete, resource-bounded, and failed analyses; only complete analyses contribute negative evidence. Edge deduplication is scoped by IR statement, value identity, access path, and source witness because same-named locals in different methods and facts from different seeds remain distinct. HapBench positive and negative cases exercise these invariants, and project isolation enforces them at corpus scale.

3.11 Scalable Execution

SDK declarations are loaded lazily into the Scene when taint analysis starts, keeping frontend construction independent of query-specific augmentation. Source and sink rules resolve against the designated SDK, and IFDS resource bounds remain explicit. Project isolation prevents cross-project frontend state, while phase-aware evidence retention excludes incomplete analyses from negative results.

Let C be the number of executable call/property candidates, d the maximum number of package/namespace-compatible rules returned by the rule index, and k_{recv} the receiver-definition bound. Identity resolution costs $O(Cdk_{recv})$ time and $O(C)$ output space; in practice d is small because package and namespace indexing precede member matching. The evidence carried by a source fact is immutable and constant-sized, so it refines fact identity without changing the underlying IFDS formulation. The post-IFDS phase is independently capped by the number of source candidates, visited methods and values, emitted outcomes, and path length. The evaluated defaults scan at most 100,000 methods and 5,000 sources, retain at most 10,000 states per source, and bound paths at 100 statements. It therefore cannot expand into an unbounded event-loop search; paths beyond these bounds are not emitted, and every retained outcome remains labeled as supplementary rather than as an IFDS proof. These bounds make the extra analyses predictable while preserving the provenance needed to distinguish the two producers.

The complete pipeline follows a fixed order: Scene construction, lifecycle modeling, report call-graph construction, import/alias extraction, API identity resolution via *Accept*, callback-augmented chain recovery, detector-local sink tracing, IFDS taint analysis from the lifecycle-aware `DummyMain`, bounded asynchronous supplementation, endpoint linkage, and evidence-graph assembly. A key property is that ArkPrism never treats a missing layer as proof of absence: if a call-chain predecessor is absent, the usage still becomes a local evidence chain; if a taint path has no compatible detector endpoint, both records remain visible but unlinked.

4 Implementation

ArkPrism is implemented in TypeScript as a command-line analyzer. It reuses ArkAnalyzer's Scene, IR, and call-graph abstractions and integrates HapFlow through a dedicated analysis layer.

Rule bases. The evaluated configuration contains 58 nonempty system-package groups and 716 detector records (710 unique package–namespace–member identities and 665 unique namespace–member identities): 453 direct, 213 manager/receiver, and 50 property records. Each record stores namespace, method/property, permission, privacy category, access mode, and optional receiver factories. The taxonomy contains 38 privacy categories and 74 distinct permission labels. Structural validation enforces required fields and access modes, while canonicalization prevents repeated configuration records from inflating reported occurrences.

Rule construction. The detector inventory is a versioned policy input rather than a set mined from ArkPrism output. Construction preserves package, namespace, member/property, permission, category, and access-mode decisions, then checks structural validity and namespace-constrained SDK migration. The evaluated taint query separately defines 257 privacy-data rules with an explicit source kind, concrete carrier, owner, and signature metadata. This separation makes the query contract reproducible: detector rules answer which configured operations occur, whereas source rules answer which typed values enter propagation.

Detector coverage and taint-source seeding use separate rule universes. The IFDS inventory admits 257 privacy-data rules whose API-20 declarations expose a concrete return or callback carrier, plus two explicitly modeled *framework_input* lifecycle rules. Other configured operations, including executable properties without a modeled property carrier, remain detector evidence rather than becoming speculative taint seeds. This carrier contract keeps permission-bearing operations visible while ensuring that every propagated fact names the value that may contain sensitive data.

The resolver treats an explicit parameter array, including the empty array, as an exact arity constraint and compares normalized parameter types and literal discriminators. This prevents a zero-argument Promise rule from absorbing a callback overload and prevents event-specific `on`/`off` rules from matching another event with the same parameter names. Module, namespace, and class are hard owner constraints: if an explicitly named class is unavailable in the loaded SDK, resolution fails instead of falling back to another class with the same member name. Against API-20, all 257 privacy-data rules resolve to exactly one declaration: there are no unresolved or ambiguous rules, signature collisions, or carrier-type mismatches. The two framework-input rules likewise resolve uniquely under both the API-17 benchmark SDK and API-20 corpus SDK. The API-17 compatibility audit resolves 249 of the 257 privacy-data rules and records the eight version-incompatible rules without substituting a different owner. The loader also rejects a missing source kind, non-concrete return carrier, callback type, or carrier index. The structural audit reports no malformed detector, source, or sink entries and 29 configured sinks. A namespace-constrained migration table maps historical modules to exports of consolidated `@kit.*` packages; it is not treated as whole-package equivalence. Detector rules, privacy-data sources, framework inputs, and SDK-resolution records remain distinct analysis inputs.

Detector. The API detector operates per Ark file: it obtains system imports, expands package aliases, normalizes the serialized access field to namespace, receiver, or property mode, and scans every method CFG. Method matching normalizes `Class.method`, `method()`, and event-style signatures while preserving namespace context. Package-compatible rules form the candidate set; indirect calls are then accepted only with an exact SDK receiver type, exact declaring-class identity from the call-target signature, an intra-procedural definition/factory-origin chain (bounded at eight definitions), or an exact namespace receiver for a non-generic member. The selected evidence is stored in `matchEvidence`; package-only and ambiguous method-only matches are rejected. For IR-lowered multi-segment APIs (e.g., `request.agent.create`), the detector records namespace-member aliases when ArkAnalyzer lowers the namespace prefix into a temporary.

Call-chain tracer. The tracer builds an enhanced reverse call map from ArkAnalyzer call-graph edges plus supplementary edges recovered from FunctionType/ClosureType signatures, class-field callback assignments, and ArkUI builder-option callbacks. Entry methods are classified through lifecycle and callback naming conventions; if no upstream entry is found, the tracer emits a local fallback chain.

Detector-local sink analysis. For each detector occurrence, the sink analyzer follows local and callback-related values and classifies observations into log, UI display, storage, network, data return, intent, and share categories. A semantic-enrichment pass adds guard conditions, try-catch context, and loop evidence. These observations are not the IFDS sink universe.

IFDS engine and asynchronous supplement. The analysis lazily loads SDK files, creates the lifecycle-aware `DummyMain` entry (Section 3.6), applies pointer analysis, and loads the independent IFDS source/sink query. The solver derives call targets from Scene and pointer-analysis evidence, including receiver-refined indirect targets; it does not reuse the separately augmented report call graph. Unknown-signature source matching retains the rule's module, namespace, class, arity, and parameter metadata. Generic members such as `on`, `get`, and `create` require receiver/module evidence; a string-dispatched overload such as `on('locationChange',...)` must additionally match the configured event literal. Thus, an unrelated `avPlayer.on('error',...)` cannot inherit a geolocation or sensor source solely from the member name and arity.

IFDS edge keys combine IR object identity, access-path state, and immutable source evidence rather than statement text, preventing distinct same-named locals or different source seeds from being merged. Normal, call, return, exceptional, callback, Promise, and `await` propagation preserve this evidence. Exception transfer is value-dependent: an `Error` payload is tainted only when one of its constructor arguments depends on the incoming fact. The solver analyzes all sources jointly; batching is an explicit resource fallback. After IFDS terminates, a separately bounded callback-/Promise scan recovers typed callback, `then`, and `await` paths that are absent from the supergraph. It seeds the configured callback-parameter index and uses structured `Value.getUses/ValueEqual` dependencies rather than textual IR-local matching; in particular, temporary `%8` does not match `%85`. Outcomes are deduplicated only when their source evidence, tainted values, and complete statement paths agree, then merged with, but not relabeled as, solver outcomes. Unknown-signature sinks likewise require method and receiver/namespace agreement, and incomplete analyses do not produce empty negative results.

Evidence linkage. Every path records whether its seed is *privacy_data* or *framework_input*, together with its resolved module, namespace, class, member, carrier, SDK signature, and rule origin. Evidence-graph assembly considers only privacy-data paths, and joins one to a detector occurrence only when project, normalized source location, and compatible API identity agree. The evidence linker rejects paths that lack source evidence or whose first normalized endpoint disagrees with that evidence. Detector-local sink observations, unlinked privacy-data paths, and framework-input paths remain separate records. This explicit partial join prevents aggregate IFDS paths from being interpreted as downstream sinks of every detector occurrence.

Lifecycle and call-graph context. The lifecycle modeler builds $\mathcal{L}$ (Section 3.6) and orders ability, component, and event entries in `DummyMain`. The default removes unbounded lifecycle back edges while allowing an explicit second firing for selected repeatable events. Separately, the report call graph augments RTA/CHA with provenance-labeled lifecycle transitions, while ArkAnalyzer's pointer analysis supplies receiver-sensitive targets to IFDS.

5 Evaluation

We answer four questions: **RQ1** How accurately does ArkPrism resolve privacy-API identities, and which evidence forms account for that accuracy? **RQ2** How accurately do ArkPrism and HapFlow classify source-to-sink flows? **RQ3** How do IR recovery, receiver refinement, lifecycle bounds, and asynchronous recovery affect analysis quality? **RQ4** What evidence and scalability patterns appear across a large ArkTS corpus? Each oracle supports a distinct claim: source-first benchmarks evaluate identity, HapBench evaluates executable flow classification, ArkAsyncBench attributes asynchronous coverage, a stratified source audit evaluates real-project path semantics, and the corpus measures prevalence and cost.

5.1 Evaluation Protocol

Comparison setup. We compare ArkPrism and HapFlow using ArkAnalyzer IR and an identical source–sink query over HapBench. HapBench is evaluated with its bundled API-17 SDK, while the real-project benchmarks and corpus use OpenHarmony API-20. Each experiment uses its designated SDK without fallback, and IFDS analyzes each project as a whole.

Identity benchmarks. *ArkSourceFirst60* fixes ten projects from each of six project-scale/configured-import strata before detector execution. A project-wide AST pass enumerates statically named candidates compatible with the fixed rule/import universe but does not label them; source review then assigns 186 exact-location decisions. The resulting oracle contains 90 sensitive occurrences in 16 projects, 96 same-name or owner-incompatible negatives, and 44 confirmed-zero projects. *ArkIdentityBench* contains 32 positive and 32 matched-negative cases spanning imports, typed and constructed receivers, factories, properties, compound namespaces, and package migration.

The *Top-120 stress audit* complements the source-first oracle by concentrating projects with high API volume. Its 1,533 reviewed location rows canonicalize to 666 project–API keys with executable source evidence. ArkPrism preserves all 666 confirmed keys, so this experiment measures confirmed-key coverage under a high-volume workload.

Flow benchmarks. *HapBench* comprises all 67 executable cases released with HapFlow: 53 positive and 14 negative. We preserve its published case oracle and additionally audit every canonical source–sink endpoint pair reported by ArkPrism. *ArkAsyncBench* contains 48 source-first cases: 24 positives and 24 matched negatives over callback (T1), Promise-`then` (T4), and `await` (T5) carriers. Positives pass the configured carrier, a carrier property, or an alias to a sink; negatives pass an error, constant, rejection value, ignored result, or independent same-named local.

Real-project path audit. A deterministic diversity sampler selects 120 of the 449 corpus paths from 48 projects: 90 privacy-data and 30 framework-input paths; 64 IFDS, 30 supplementary, and 26 dual-provenance paths; and 23 short, 70 medium, and 27 long paths. The sample includes every network and UI path stratum. Structured source and IR review judges source identity, sink identity, explicit value dependence, reachability, and provenance; a path is correct only when all five dimensions hold.

Large corpus. The corpus contains 1,014 ArkTS projects analyzed under a common implementation, API-20 SDK, rule set, and resource policy. Its applications, libraries, and capability demonstrations expose diverse project roles and API usage patterns. Each project contributes one validated result; aggregation excludes incomplete analyses, unresolved endpoints, and invalid provenance.

Metrics. Identity occurrences match on project, normalized file, line/column, canonical package/namespace/member, and access kind. Flow benchmarks report precision, recall, specificity, balanced accuracy, F1, and MCC. We use Wilson intervals for binomial rates and exact two-sided McNemar tests for paired predictions. Corpus statistics quantify project prevalence, concentration (Gini and HHI), Spearman associations, source scope, clone sensitivity, and runtime; the preceding oracles provide the accuracy evidence.

5.2 RQ1: Privacy-API Identity

Source-first accuracy. Within the fixed rule set and statically enumerable candidate universe, ArkPrism reports exactly the 90 gold occurrences in ArkSourceFirst60: TP/FP/FN= 90/0/0, for 100% observed precision, recall, and F1 (95% Wilson interval: 95.91–100%). It also classifies all 16 positive and 44 zero projects correctly. Exact-location matching prevents one call from masking a missed occurrence elsewhere in the same file.

The gold set contains 44 executable namespace properties, 26 namespace calls, 11 typed receivers, four constructed receivers, three manager receivers, and two factory receivers. Thus, 20/90 occurrences require receiver evidence beyond the accepted line's namespace syntax, and nearly half are property reads rather than calls. The 96 negatives stress the complementary boundary: application wrappers, UI controllers, collections, test drivers, and third-party methods reuse generic members such as `create`, `on`, `request`, and `getData`.

Evidence progression. Table 3 shows how the decision boundary changes. Lexical matching reaches every positive but accepts every matched collision. Import compatibility removes these collisions; receiver and factory evidence then recover indirect uses. The complete resolver adds constructed receivers, executable properties, compound aliases, and namespace-constrained migration, classifying all 64 cases correctly.

Table 3. Identity evidence progression (rates in %, except MCC).

Configuration	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1	Acc.	MCC
Lexical token	32	0	32	0	50.00	100.00	0.00	66.67	50.00	0.000
Import-aware namespace	17	32	0	15	100.00	53.13	100.00	69.39	76.56	0.601
Declared receiver	26	32	0	6	100.00	81.25	100.00	89.66	90.63	0.827
Factory-aware receiver	28	32	0	4	100.00	87.50	100.00	93.33	93.75	0.882
ArkPrism	**32**	**32**	**0**	**0**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**1.000**

Table 4. HapBench case-level classification (rates in %, except MCC).

Tool	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1	Acc.	BAcc.	MCC
HapFlow [15]	51	10	4	2	92.73	96.23	71.43	94.44	91.04	83.83	0.717
ArkPrism	50	**14**	**0**	3	**100.00**	94.34	**100.00**	**97.09**	**95.52**	**97.17**	**0.881**

High-volume reproduction. ArkPrism recovers all 666 manually confirmed project–API keys in the Top-120 stress audit (100% confirmed-key coverage; 95% Wilson lower bound 99.43%), including 558 namespace-qualified, 102 member/receiver, five template-expression, and one compound-qualified key. This result demonstrates stable recognition across both dominant namespace calls and less common receiver, template, and compound forms under high API volume.

Answer to RQ1. On a source-first real-project oracle defined over the fixed rule and static-candidate universe, ArkPrism resolves all 90 sensitive occurrences and rejects all 96 candidate collisions. The controlled benchmark attributes this result to the composition of package, receiver, factory, property, and IR evidence; the stress audit adds 102 confirmed member/receiver keys and shows that the evaluated detector configuration preserves all 666 previously confirmed high-volume keys.

5.3 RQ2: End-to-End Flow Classification

Table 4 reports the complete HapBench reproduction. ArkPrism classifies 64/67 cases correctly (50 TP, 14 TN), while HapFlow classifies 61/67 (51 TP, 10 TN). ArkPrism rejects every negative case and retains 50/53 positives, yielding 100% precision and specificity, 97.09% F1, 97.17% balanced accuracy, and 0.881 MCC.

The tools disagree on nine cases: ArkPrism alone is correct on six and HapFlow alone on three (exact paired $p = 0.508$). The observed advantage is concentrated in negative-case discrimination. Figure 4(a) shows Wilson uncertainty, and panel (b) localizes every error across seven categories and five configurations. ArkPrism reaches 100% F1 in six categories and 85.71% in lifecycle cases.

The endpoint-pair audit tests precision inside positive cases. The 52 raw paths canonicalize to 51 source–sink pairs: 50 match the benchmark behavior, while one additional virtual target appears inside a positive case. With three missing positive endpoints, pair-level TP/FP/FN is 50/1/3, giving 98.04% precision, 94.34% recall, and 96.15% F1.

Where the classifications differ. ArkTS-specific IR recovery resolves the anonymous initializer and exceptional successor missed by HapFlow, while constant-index access paths, receiver-refined dispatch, and bounded lifecycle transitions remove its array, virtual-target, and cross-event false positives. The three published positives not emitted by ArkPrism require, respectively, an implicit parent lifecycle execution, state continuity across framework-created extension instances, or a later ability creation after a click callback. Appendix A.2 evaluates this event-order distinction without changing the primary published oracle.

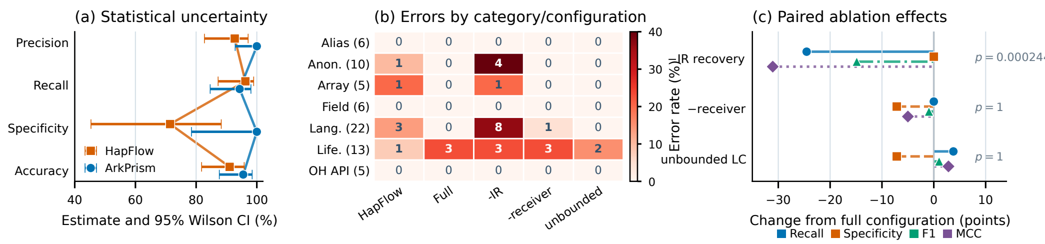

Fig. 4. HapBench uncertainty, category errors, and paired ablation effects.

Table 5. HapBench ablation results (rates in %, except MCC).

Configuration	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1	Acc.	BAcc.	MCC
ArkPrism	50	**14**	**0**	3	**100.00**	94.34	**100.00**	97.09	95.52	**97.17**	0.881
−IR recovery	37	**14**	**0**	16	**100.00**	69.81	**100.00**	82.22	76.12	84.91	0.571
−receiver refinement	50	13	1	3	98.04	94.34	92.86	96.15	94.03	93.60	0.831
Unbounded lifecycle	**52**	13	1	**1**	98.11	**98.11**	92.86	**98.11**	**97.01**	95.49	**0.910**

Answer to RQ2. ArkPrism combines 100% case-level precision and specificity with 97.09% F1 on the full executable suite. Its principal empirical advantage over HapFlow is precise rejection of negative cases; the endpoint audit retains 98.04% precision at the finer source–sink-pair unit.

5.4 RQ3: Mechanism and Path Quality

IR, receiver, and lifecycle mechanisms. Table 5 reports pre-specified ablations under the same query. Removing IR recovery loses 13 additional positives; the full configuration is exclusively correct on 13 cases and never exclusively wrong ($p = 0.000244$, significant after Bonferroni correction for four tests). Receiver refinement removes one infeasible virtual target without reducing recall.

The bounded default preserves all 14 negatives and attains 97.17% balanced accuracy, compared with 95.49% under unbounded lifecycle cycling. Removing the bound accepts two additional published positives but also recreates the benchmark's explicit unreachable-flow negative. Because F1 omits true negatives, it ranks the unbounded setting higher on that narrower objective; balanced accuracy measures the positive and negative obligations together. Under the strict executed-order contract in Appendix A.2, bounded ArkPrism classifies 67/67 cases, whereas unbounded cycling creates three cross-event false positives. The default therefore targets the audit objective: preserve negative-case discrimination unless an executable transition witness exists.

Asynchronous provenance. HapBench exercises callbacks with explicit IFDS edges; ArkAsyncBench targets complementary continuation gaps. Table 6 shows that T1 callback and T5 `await` paths receive corroborating evidence from both analyses, while all nine T4 Promise-`then` positives are supplement-exclusive. Removing supplementation therefore preserves 15 core-supported positives but misses the nine T4 cases ($p = 0.00390625$), with no change among the 24 negatives.

A separate output-guided diagnostic over 25 callback-heavy projects confirms all ten selected real-code asynchronous paths: one Promise-`then` path carrying an advertising identifier and nine `await` paths carrying media metadata or reverse-geocoding values. This diagnostic tests explicit carrier dependence in executable project code; ArkAsyncBench supplies the controlled evidence for T1 callback binding.

Stratified semantic audit. Table 7 reports the 120-path source/IR review. Source identity, sink identity, and provenance are correct for every path. All 90 sampled privacy-data paths are fully

Table 6. ArkAsyncBench classification and provenance attribution.

(a) Classification

Construct	Pos.	Neg.	Full TP	Core-only TP	Unique supp. TP	TN
T1 callback	9	9	9	9	0	9
T4 then	9	9	9	0	**9**	9
T5 await	6	6	6	6	0	6
Total	24	24	24	15	**9**	24

(b) Provenance and real-code audit

Construct	IFDS	Supp.	Both	Supp.-only	Audit
T1 callback	9	9	9	0	–
T4 then	0	9	0	**9**	1/1
T5 await	6	6	6	0	9/9
Total	15	24	15	**9**	10/10

Table 7. Manual semantic audit of 120 real-project paths.

Audited dimension	All	Privacy	Framework	IFDS	Supp.	Both	Rate	95% CI
Source identity	120/120	90/90	30/30	64/64	30/30	26/26	100.00%	[96.90, 100]
Sink identity	120/120	90/90	30/30	64/64	30/30	26/26	100.00%	[96.90, 100]
Explicit value dependence	114/120	90/90	24/30	58/64	30/30	26/26	95.00%	[89.52, 97.69]
Reachability	114/120	90/90	24/30	58/64	30/30	26/26	95.00%	[89.52, 97.69]
Provenance label	120/120	90/90	30/30	64/64	30/30	26/26	100.00%	[96.90, 100]
Complete path	114/120	**90/90**	24/30	58/64	**30/30**	**26/26**	95.00%	[89.52, 97.69]

Table 8. Privacy evidence across 1,014 projects.

Evidence	Records	Projects	Project rate	Rec./positive proj.	Prod.	Test	Gen.
Privacy-API usages	2,336	353	34.81%	6.62	–	–	–
Detector-local sinks	1,754	224	22.09%	7.83	–	–	–
Privacy-data paths	243	29	2.86%	8.38	231	2	10
Framework-input paths	206	159	15.68%	1.30	49	157	0
All configured paths	449	181	17.85%	2.48	280	159	10

correct, including all 30 supplement-only and 26 dual-provenance paths in the full sample. Overall path precision is 114/120 (95.00%; 95% Wilson interval 89.52–97.69%).

The six rejected paths all belong to the optional framework-input model. Four join a lifecycle-written static URI field to unrelated component fields or constants; two connect a lifecycle argument to a permission Toast or a hard-coded event value. This localization matters operationally: the core privacy-data analysis attains 90/90 in the audit, while the framework-input layer remains separately identifiable by source kind and provenance instead of silently entering the privacy-path claim.

Answer to RQ3. IR recovery supplies the largest statistically supported coverage gain, receiver refinement and lifecycle bounds preserve negative-case discrimination, and asynchronous supplementation uniquely recovers nine Promise-`then` cases. The real-project audit confirms every sampled privacy-data path and measures 95.00% precision across the broader path universe.

5.5 RQ4: Large-Corpus Characterization

The corpus analysis covers 31,205 ArkTS/TypeScript files and 233,894 methods. Table 8 separates detector evidence, privacy-data paths, and framework-input paths. All 449 configured-query paths retain source and sink endpoints; source scope further distinguishes production, test, and generated code.

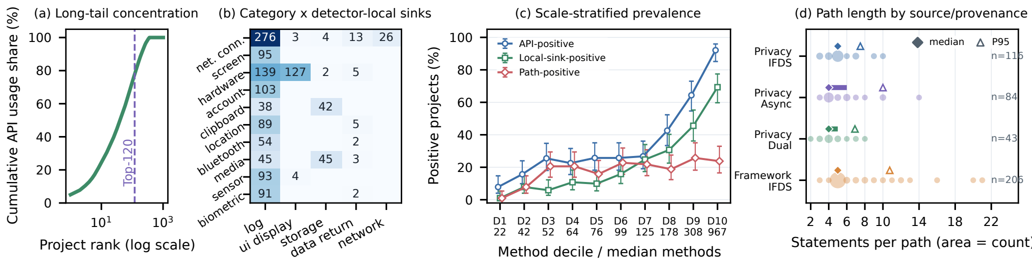

Fig. 5. Corpus evidence concentration, category–sink structure, scale-stratified prevalence, and provenance-conditioned path complexity.

The scope split exposes two distinct corpus phenomena. Privacy-data paths overwhelmingly occur in production source (231/243), while framework-input paths are dominated by generated test abilities (157/206). We therefore use the former for privacy-flow claims and retain the latter as a separately labeled analysis of framework-provided values.

Concentration and structure. API evidence is long-tailed: 65.19% of projects have no recognized API, while the top 10, 50, and 120 projects contain 24.49%, 54.92%, and 77.83% of all occurrences. The API Gini coefficient is 0.857; HHI is 0.0104, equivalent to 95.8 equally sized contributing projects. Network connectivity (18.28%), screen identity (17.25%), hardware identity (15.11%), account data (8.05%), and clipboard data (5.09%) form the five largest categories.

Project size helps explain identity volume but not path formation. Method count correlates moderately with API occurrences ($\rho = 0.485$) and weakly with configured paths ($\rho = 0.164$); API occurrences correlate strongly with detector-local sinks ($\rho = 0.789$) and weakly with configured paths ($\rho = 0.175$). Figure 5(c) partitions projects into ten equally sized method-count strata and reports Wilson intervals. From the smallest to largest stratum, API-positive rates rise from 7.84% to 92.08% and detector-local-sink-positive rates from 0.98% to 69.31%. Path-positive rates instead remain between 18.81% and 25.74% across the four largest strata, separating codebase scale from configured-path formation.

Destinations and paths. Logging accounts for 1,441/1,754 detector-local sinks (82.16%), followed by UI display (136; 7.75%), storage (94; 5.36%), data return (57; 3.25%), and network (26; 1.48%). Call chains have median/P95 lengths of 1/4; configured paths have median/P95 lengths of 5/10 and a maximum of 21 statements. Provenance comprises 322 IFDS, 84 supplementary, and 43 dual paths. Figure 5(d) exposes their discrete structure: privacy IFDS, supplementary, and dual paths have medians of 5, 4, and 4 statements and P95 values of 7.5, 10, and 6.9, respectively; framework-input IFDS paths have a median of 5 and the longest tail (P95 10.75; maximum 21).

Integrity and cost. All 1,014 projects complete without IFDS batching, resource or callback truncation, timeout, or out-of-memory failure. The audit resolves all 3,316 trace endpoints, validates all 236 detector-to-path links, and finds no invalid source kind or provenance. The solver processes 3.16 million IFDS edges in total; per-project median/P95/maximum is 805/11,396/197,173. Deduplication reduces 492 raw outcomes to 449 paths. Median runtime is 20.2 s, P95 is 26.9 s, and the 10,757-method outlier completes in 577.9 s.

Clone sensitivity. The token profile contains 31,998 source files and 21,430 unique fingerprints. It forms nine exact-duplicate and eleven near-duplicate project clusters. Retaining one representative per exact cluster changes API/sink/path-positive rates from 34.81/22.09/17.85% to 35.18/22.31/17.99%; near-cluster representatives yield 35.05/22.26/18.03%. The maximum shift is 0.36 percentage points, showing that project-level rates are not driven by repeated copies of a small template set.

Answer to RQ4. ArkPrism completes all 1,014 projects without resource fallbacks. Privacy-data paths are primarily production-source phenomena, identity and sink evidence are strongly concentrated, and clone-aware resampling leaves all project rates stable within 0.36 percentage points.

6 Discussion

6.1 Evidence as the Analysis Contract

ArkPrism treats source resolution as an evidence-producing analysis rather than a configuration side effect. Every accepted API identity carries a package/receiver/access witness; every query source carries a typed value; and every path records its producing analysis. The preserved unit $\langle identity, witness, carrier \rangle$ uses $carrier = \perp$ for detector-only operations. This contract separates three questions that privacy tools often collapse: whether a configured API occurs, whether a modeled value may reach a configured sink, and whether that behavior violates a policy. ArkPrism answers the first two and supplies the source, sink class, permission, call context, and provenance needed to review the third.

The empirical results follow the same structure. ArkSourceFirst60 and ArkIdentityBench evaluate exact-location identity decisions. HapBench evaluates case and endpoint-pair flow classification. ArkAsyncBench isolates continuation recovery, while the 120-path source audit tests real-project path semantics. The Top-120 set stresses recovery of dense, previously confirmed identity evidence. The corpus then measures prevalence, concentration, traceability, and cost without treating tool output as a corpus-wide oracle.

6.2 Core Privacy Paths and Framework Context

The semantic audit provides a useful boundary. All 90 sampled privacy-data paths are correct, including IFDS, supplementary, and dual-provenance results. The six rejected paths occur only when the optional framework-input model composes lifecycle values with component or event context: four confuse unrelated static/component fields, and two retain control co-occurrence without explicit value dependence. Because source kind is part of every record, these candidates remain outside the core privacy-data result and can be filtered without changing API identity or privacy-carrier propagation.

This boundary also clarifies corpus interpretation. Of 243 privacy-data paths, 231 originate in production source; by contrast, 157/206 framework-input paths originate in generated test abilities. Reporting the two source universes separately exposes this structural difference and gives auditors a direct way to prioritize production privacy evidence.

6.3 Precision-Oriented Reachability

Lifecycle modeling illustrates why the evaluation emphasizes specificity and balanced accuracy in addition to recall and F1. Unbounded cycling accepts more cross-event positives under the published HapBench labels, but also creates the suite's explicit unreachable-flow false positive. F1 ignores the lost true negative; balanced accuracy accounts for both classes and favors the bounded policy. Under the executed-order sensitivity contract, the bounded policy classifies all 67 cases, whereas unbounded cycling creates three false positives. The default therefore requires an explicit parent, same-instance, or later-creation witness before connecting lifecycle phases.

Asynchronous supplementation follows the same principle. It is bounded independently from IFDS and does not erase its origin. T1 callback and T5 `await` paths can be corroborated by both analyses; T4 Promise-`then` supplies the benchmark's nine unique increments. Provenance makes this composition inspectable rather than presenting every recovered path as an equivalent solver proof.

6.4 Validity Controls

Oracle separation. ArkSourceFirst60 is selected by project-scale and configured-import strata before detector execution. Its project-wide candidate enumeration locates code but does not assign labels, and the final decision record retains exact source locations for all 186 candidates. The resulting recall claim is explicitly scoped to the fixed rule and statically enumerable candidate universe. ArkAsyncBench similarly defines positives and matched negatives from source-level value dependence. The Top-120 set serves a complementary role by testing preservation of 666 confirmed keys under high API volume, including 102 member/receiver keys.

Configuration control. HapFlow and ArkPrism use the same API-17 SDK and source–sink query on HapBench, while the real-project evaluation uses OpenHarmony API-20 without SDK fallback. Versioned rules, SDK declarations, and analyzer builds define each evaluation configuration; pre-aggregation validation checks endpoint links, source kinds, provenance, and completion state.

Query scope. The detector and taint query are fixed, versioned inputs. ArkSourceFirst60 evaluates configured identities expressible by its static candidate enumerator; APIs absent from the policy, computed members outside that enumerator, and dynamically loaded code are outside this metric. The taint query additionally requires a concrete return, callback, or explicit framework-input carrier, so detector-only properties do not enter the privacy-path denominator. This scope keeps identity, seeding, and path claims aligned with the implemented contract.

Corpus composition. The corpus includes applications, libraries, demonstrations, tests, and a small amount of generated source. Results therefore characterize the analyzed collection. Source-scope stratification prevents generated test abilities from being mistaken for production privacy paths. Exact- and near-clone resampling changes project rates by at most 0.36 percentage points, showing that the principal corpus patterns are not driven by repeated project copies.

6.5 Applicability

ArkPrism is a source-based static analysis for ArkTS projects. Native C/C++ bridges, runtime-loaded code, server-side behavior, and consent state require complementary evidence such as HarmoBridge [18] or runtime monitoring. Within its stated contract, ArkPrism produces reviewable API identities and may-paths rather than automatic legal conclusions.

7 Related Work

7.1 Source Identity and Privacy API Recognition

Accurate source/sink specifications are critical for mobile privacy analysis. Android studies have mined permission mappings and source/sink lists via Stowaway, PScout, SuSi, Axplorer, and API-documentation analyses [6, 7, 21, 43, 45], and audited privacy behavior by connecting code, policies, and permissions [3, 55, 62, 63]. These works show that source/sink definitions encode platform semantics and strongly affect reported results.

ArkPrism applies this principle to OpenHarmony, where package migration from `@ohos.*` to consolidated `@kit.*` exports compounds the challenge. Its matching function separates API identity from exact package spelling while retaining namespace constraints. HapFlow's `Json2ArkMethodSignature` loader resolves namespace/method entries to SDK declarations, but its rule schema does not encode executable fields or detector-side factory-origin witnesses [15]. ArkPrism complements signature resolution with the receiver evidence needed for manager methods and an evidence-carrying identity layer for privacy analysis.

7.2 Asynchronous Semantics and Lifecycle Modeling

Mobile analysis must model lifecycle, callbacks, and implicit edges precisely. Android ICC modeling includes ComDroid, CHEX, Epicc, IC3, IccTA, DidFail, and RAICC [19, 29, 31, 36, 38, 39, 48]; EdgeMiner recovers implicit framework callbacks [13]. Hybrid-app analyses bridge Java, JavaScript, and WebView semantics [11, 30, 57, 58]. These studies show that missing implicit edges dominate mobile analysis errors.

OpenHarmony has analogous but different callback problems: ArkUI declarative builders, class-field callbacks, FunctionType/ClosureType values, and Promise handlers create edges absent from ordinary calls. WEMINT [54] demonstrates explicit callback modeling for asynchronous mini-program APIs via taint rules; ArkPrism differs by resolving callbacks from ArkAnalyzer's FunctionType/ClosureType IR and by handling ArkUI builder-option and receiver-factory semantics. HapFlow handles closure semantics [15]; ArkPrism adds a bounded, separately provenance-labeled post-IFDS recovery for unresolved callback, Promise `.then()`, and `await` paths, plus report-CG edges for field and builder callbacks. The lifecycle state machine (Section 3.6) captures ability/component phase transitions that the flat DummyMain cannot express.

7.3 Evidence Representation and Large-scale Auditing

Android taint analysis has a rich history. TaintDroid [20] motivated smartphone privacy monitoring; FlowDroid [5] introduced context-/flow-/field-/object-sensitive lifecycle-aware analysis; Amandroid, DroidSafe, IccTA, LeakMiner, and AndroidLeaks explored inter-procedural data-flow, vetting, and market-side detection [25, 26, 31, 52, 61]; HornDroid and COVERT added formal and compositional perspectives [8, 12]. More recently, PacDroid unifies pointer analysis, inter-procedural propagation, ICC, and lifecycle handling and evaluates the resulting soundness–precision–performance tradeoff against several Android frameworks [16]. These systems are important methodological references, but they consume Android bytecode and Android framework models; running them on ArkTS would change both the input language and the analysis query. We therefore do not present their published numbers as an empirical baseline for ArkPrism.

ArkPrism uses HapFlow as the directly comparable OpenHarmony taint baseline because both accept ArkTS projects and answer the same source-to-sink reachability query. ArkAnalyzer and its context-sensitive Pointer Analysis Kit (APAK) provide the source IR, call graphs, and points-to information on which a taint client can be built [14, 60], but do not themselves emit privacy source-to-sink findings. HarmonyFlow instead analyzes compiled Ark Panda IR and evaluates pointer/call-edge recovery [51]; its input representation and output query make it a complementary front-end reference rather than an end-to-end baseline. TaintMini's Universal Data Flow Graph [33] is a core analysis representation that drives cross-page data-flow resolution in mini-programs; ArkPrism's provenance-preserving evidence graph is assembled from detection, call-chain, sink, and taint results while retaining their distinct producers.

Malware and behavior research uses permissions, API calls, graphs, and models to classify malicious apps [1, 4, 22, 24, 40, 44, 49, 53, 56, 59, 64–66]; AndroZoo enables ecosystem-scale studies [2]. ArkPrism follows this empirical principle: its evidence graph is more structured than a flat feature vector because a result retains source snippets, call contexts, sink classes, and metadata.

ArkPrism builds on IFDS/IDE foundations [46, 47] and reusable infrastructures (Soot, WALA, Heros [9, 23, 50]). ArkAnalyzer plays this role for ArkTS [14]. APAK shows that ArkTS closure and framework modeling materially affects downstream call targets and false positives relative to CHA/RTA [60]; ArkPrism consumes this pointer evidence in receiver-sensitive source and sink resolution. Other OpenHarmony efforts explore testing [17, 35], repair [34], and compiled-IR analysis [51], showing that OpenHarmony requires ecosystem-specific analysis [27, 32, 41].

ArkPrism follows the same layered principle: reuse the framework, then add domain-specific recognition and provenance-preserving evidence-graph mapping.

7.4 Evaluation Methodology for Mobile Analyses

Comparative studies show that mobile-analysis results are easily distorted by different source/sink lists, different queries, incomplete benchmark subsets, and undocumented ground truth. ReproDroid evaluates six Android taint analyzers under a common query and execution framework [42]. TaintBench goes further by storing expected and unexpected real-world flows in a machine-readable format and validating labels through multi-expert review [37]. Mutation-based evaluation complements fixed benchmarks by systematically exposing unsound analysis choices [10]. These studies shape our protocol: we use the complete HapBench suite, keep the source and sink query fixed, report both positive- and negative-case metrics, and analyze every disagreement. ArkSourceFirst60 supplies source-first occurrence-level accuracy within a fixed rule and static-candidate universe, while the Top-120 audit tests confirmed-key preservation under high API volume.

8 Conclusion

ArkPrism makes privacy-source identity a first-class stage of ArkTS static analysis. Package migration, receiver origin and type, executable properties, compound namespaces, and IR aliases establish which configured platform API occurs; well-formed carriers and immutable witnesses preserve how modeled data enters IFDS or bounded continuation recovery. Within its fixed rule and static-candidate universe, this contract resolves all 90 exact-location occurrences in the source-first benchmark and classifies all 64 matched identity cases. It combines 100% case-level precision and specificity with 97.09% F1 on HapBench, while ArkAsyncBench isolates nine continuation-exclusive Promise-`then` paths. The real-project audit confirms all 90 sampled privacy-data paths and 95% of the broader path sample. Across 1,014 projects, ArkPrism identifies 2,336 privacy-API usages and 243 privacy-data paths while retaining source, sink, and provenance evidence for every result. Together, these results establish evidence-carrying source resolution as a practical foundation for precise ArkTS privacy auditing.

Data Availability

The replication package contains the ArkPrism source and build scripts; API, source, sink, and package-migration rules; ArkSourceFirst60 and ArkIdentityBench; the Top-120 stress audit; HapBench and ArkAsyncBench oracles and outputs; the 120-path semantic review; and per-project results for the 1,014-project corpus. Machine-readable results regenerate every table and figure. Dataset redistribution follows each constituent project's license; project identifiers and derived reports are supplied where source redistribution is restricted.

A Reproduction Details

A.1 HapBench Case-level Outcomes

Table 9 lists every error produced by either executable tool under the fixed 67-case oracle. Cases absent from the table are classified correctly by both tools. We retain the source-annotated oracle even where a more restrictive event or object-lifetime semantics would suggest a different label.

The two tools agree on 58 outcomes. Of the nine discordant cases, ArkPrism alone is correct on six and HapFlow alone on three. The exact two-sided McNemar test is therefore based on the discordant pair $(6, 3)$ and yields $p = 0.508$.

At the finer endpoint-pair unit, ArkPrism's 52 raw paths canonicalize to 51 source–sink pairs. Source annotations or direct source/IR review confirm 50; the additional pair is the `NoLeak.log`

Table 9. Case-level error inventory for the main HapBench comparison.

Tool	Out.	Category	Case	Observed mechanism
HapFlow	FN	Anonymous	`AnonymousClass2`	Anonymous instance initializer omitted
HapFlow	FN	Language	`Exceptions4`	Exceptional successor/handler not recovered
HapFlow	FP	Array	`ArrayIndexNoLeak`	Distinct constant indices collapse
HapFlow	FP	Language	`VirtualDispatch2`	Infeasible receiver target retained
HapFlow	FP	Language	`VirtualDispatch3`	Infeasible receiver target retained
HapFlow	FP	Lifecycle	`UnreachableFlow`	Lifecycle cycle creates reachability
ArkPrism	FN	Lifecycle	`ActivityLifecycle4`	Parent source method is bypassed by child override
ArkPrism	FN	Lifecycle	`BackupExtensionAbility`	Oracle assumes state across extension instances
ArkPrism	FN	Lifecycle	`Button1`	Oracle assumes click-written static state reaches later creation

Table 10. Sensitivity to strict lifecycle execution order (rates in %, except MCC).

Tool	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1	Acc.	BAcc.	MCC
HapFlow [15]	48	10	7	2	87.27	96.00	58.82	91.43	86.57	77.41	0.622
ArkPrism	50	17	0	0	100.00	100.00	100.00	100.00	100.00	100.00	1.000

target at line 14 of `VirtualDispatch1`, whose allocation receives an empty constant. The retained audit records every pair, its source identity, endpoint lines, provenance, review mode, and decision; an integrity test reproduces TP/FP/FN= 50/1/3 and 98.04% precision.

A.2 Lifecycle-Oracle Sensitivity

The primary evaluation preserves all published/source-annotated HapBench labels. We additionally define a strict executed-order contract for the three disputed lifecycle cases: an override does not execute its parent body without an explicit `super` call; an instance field does not cross framework-created instances without a same-object witness; and a UI callback occurring after initial ability creation does not feed an earlier `onCreate` without an explicit later-creation edge. This contract relabels `ActivityLifecycle4`, `BackupExtensionAbility`, and `Button1` as negative and changes no other case.

The tools then disagree on nine cases, all classified correctly only by ArkPrism; the exact two-sided McNemar test gives $p = 0.00390625$. This secondary sensitivity contract does not replace the published oracle; it makes the object and event-order semantics responsible for the difference explicit. Both tables are derived from the same case predictions.

A.3 Uncertainty and Category Robustness

Table 11 reports Wilson intervals for the main configurations. The wide specificity intervals are a direct consequence of having only 14 negative cases; balanced accuracy and MCC are included to make this class imbalance visible.

A.4 Source-first Benchmark Protocol

ArkSourceFirst60 is selected before detector execution. Projects are partitioned by source-tree size and by whether their imports are compatible with the sensitive-API rules; deterministic ordering selects ten projects from each of the six strata. Candidate discovery collects the project-wide SDK-import universe and enumerates statically named calls and property reads compatible with that universe, including static element access, named-import calls, and simple callable aliases. It

Table 11. HapBench uncertainty and category robustness.

Tool	Prec. 95% CI	Rec. 95% CI	Spec. 95% CI	Acc. 95% CI	BAcc.	MCC	Macro F1	Worst F1
HapFlow [15]	[82.7, 97.1]	[87.2, 99.0]	[45.4, 88.3]	[81.8, 95.8]	83.83	0.717	95.73	88.89
ArkPrism	[92.9, 100.0]	[84.6, 98.1]	[78.5, 100.0]	[87.6, 98.5]	97.17	0.881	97.96	85.71
−IR recovery	[90.6, 100.0]	[56.5, 80.5]	[78.5, 100.0]	[64.7, 84.7]	84.91	0.571	87.07	66.67
−receiver refinement	[89.7, 99.7]	[84.6, 98.1]	[68.5, 98.7]	[85.6, 97.7]	93.60	0.831	97.57	85.71
Unbounded lifecycle	[90.1, 99.7]	[90.1, 99.7]	[68.5, 98.7]	[89.8, 99.2]	95.49	0.910	98.81	91.67

Table 12. Per-category HapBench errors and F1.

Category	Cases	HapFlow err.	HapFlow F1	ArkPrism err.	ArkPrism F1	ΔF1
Aliasing	6	0	100.00	0	100.00	0.00
Anonymous constructs	10	1	93.33	0	100.00	+6.67
Array-like structures	5	1	88.89	0	100.00	+11.11
Field/object sensitivity	6	0	100.00	0	100.00	0.00
General language features	22	3	91.89	0	100.00	+8.11
Lifecycle modeling	13	1	96.00	3	85.71	−10.29
OpenHarmony-specific APIs	5	0	100.00	0	100.00	0.00
Total / macro	67	6	95.73	3	97.96	+2.23

neither reads a ArkPrism report nor assigns a gold label. The resulting recall metric is therefore defined over this fixed rule and static-candidate universe rather than over APIs absent from the policy or dynamically computed members.

Source review accepts an occurrence only when its executable context establishes the configured package/namespace/member and access kind through an import, SDK receiver type, constructor, manager/factory definition, or executable property read. It rejects comments, strings, declarations, same-name application methods, and unrelated library/controller receivers. The gold set contains 90 accepted occurrences; its final decision record preserves all 186 judgments, including 96 receiver- or owner-incompatible negatives. Validation checks every source location and canonical rule key and rejects duplicate occurrence identities. Evaluation joins detector and gold records on project, normalized file, exact line and column, canonical API, and access kind.

A.5 High-output Stress-audit Protocol

The 120-project stress audit labels project–namespace–member keys. A key is accepted only when executable source establishes the configured API through an imported namespace/package alias, a manager/helper definition chain, an SDK receiver type, or an executable property rule. Comments, strings, declarations, unrelated same-name methods, and ambiguous method-only matches are rejected. Each accepted key retains project, normalized package/namespace/member, access kind, source file/line, and a bounded source excerpt.

The construction set contains 1,533 reviewed location rows that canonicalize to 666 confirmed project–API keys. The companion integrity audit resolves every key against the delivered source tree and evaluated rules, verifies file/line bounds and snippets, and checks package/namespace/member compatibility. All 666 records pass. Because the set begins from an earlier output ranking, it is not an exhaustive occurrence oracle.

The detector configuration used for the corpus recovers all 666 confirmed keys in all 120 projects. It also reports 182 keys outside the reviewed set. The evaluator therefore defines *confirmed-key reproduction coverage* (666/666); unreviewed keys do not enter either side of this metric.

A.6 Real-project Path-audit Protocol

The path audit samples the complete set of 449 corpus paths. A deterministic sampler balances source kind, provenance, sink family, path-length bin, and project diversity, producing 120 records from 48 projects. The review queue contains 90 privacy-data and 30 framework-input paths; 64 IFDS, 30 supplementary, and 26 dual-provenance paths; and all available network and UI strata. Each record retains endpoint locations, path statements, source identity, and provenance.

Each referenced source/IR path receives five Boolean decisions: source identity, sink identity, explicit value dependence, reachability consistency, and provenance label. A path is accepted only when all five hold. The final audit confirms source identity, sink identity, and provenance for 120/120 paths; explicit dependence, reachability, and complete-path correctness hold for 114/120. All 90 privacy-data paths are complete. The six rejected framework-input paths consist of four unrelated static/component-field joins and two lifecycle/event joins without explicit value dependence.

A.7 Result Traceability

The replication package provides ArkSourceFirst60's 186-item decision set and occurrence gold set; the 64-case ArkIdentityBench oracle; machine-readable oracles and outputs for all 67 HapBench and 48 ArkAsyncBench cases; the Top-120 reproduction results; and the complete semantic-path audit. These records derive confusion matrices, construct/category metrics, endpoint-pair outcomes, paired tests, uncertainty intervals, and runtimes. Figure 4 and Figure 5 are generated from evaluator JSON rather than transcribed values.

Corpus tables and figures use one validated result per project under a common analyzer build, SDK, rule set, and resource policy. Aggregation checks project inventory, completion state, source and sink endpoints, provenance, and detector-to-path links. Companion JSON records every plotted value and its evaluation configuration.

References

[1] Yousra Aafer, Wenliang Du, and Heng Yin. 2013. DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android. In *Proceedings of the International Conference on Security and Privacy in Communication Systems*. 86–103.

[2] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting Millions of Android Apps for the Research Community. In *Proceedings of the 13th International Conference on Mining Software Repositories (ICMSR)*. ACM, New York, NY, USA, 468–471.

[3] Benjamin Andow, Samin Yaseer Mahmud, Justin Whitaker, William Enck, Bradley Reaves, Kapil Singh, and Serge Egelman. 2020. Actions Speak Louder than Words: Entity-Sensitive Privacy Policy and Data Flow Analysis with PoliCheck. In *29th USENIX Security Symposium (USENIX Security)*. USENIX Association, Online, 985–1002.

[4] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, Konrad Rieck, and CERT Siemens. 2014. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. In *Proceedings of the Network and Distributed System Security Symposium*.

[5] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery, New York, NY, USA, 259–269.

[6] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. 2012. PScout: Analyzing the Android Permission Specification. In *Proceedings of the ACM Conference on Computer and Communications Security*. 217–228.

[7] Michael Backes, Sven Bugiel, Erik Derr, Patrick McDaniel, Damien Octeau, and Sebastian Weisgerber. 2016. On Demystifying the Android Application Framework: Re-Visiting Android Permission Specification Analysis. In *Proceedings of the USENIX Security Symposium*. 1101–1118.

[8] Hamid Bagheri, Alireza Sadeghi, Joshua Garcia, and Sam Malek. 2015. COVERT: Compositional Analysis of Android Inter-App Permission Leakage. In *Proceedings of the IEEE/ACM International Conference on Software Engineering*. 866–877.

[9] Eric Bodden. 2012. Inter-procedural Data-flow Analysis with IFDS/IDE and Soot. In *Proceedings of the ACM SIGPLAN International Workshop on State of the Art in Java Program Analysis*. 3–8.

[10] Richard Bonett, Kaushal Kafle, Kevin Moran, Adwait Nadkarni, and Denys Poshyvanyk. 2018. Discovering Flaws in Security-Focused Static Analysis Tools for Android Using Systematic Mutation. In *Proceedings of the 27th USENIX Security Symposium*. 1263–1280. https://www.usenix.org/conference/usenixsecurity18/presentation/bonett

[11] Achim D. Brucker and Michael Herzberg. 2018. BabelView: Evaluating the Impact of Code Injection Attacks in Mobile Webviews. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*. 25–46.

[12] Stefano Calzavara, Ilya Grishchenko, and Matteo Maffei. 2016. HornDroid: Practical and Sound Static Analysis of Android Applications by SMT Solving. In *Proceedings of the IEEE European Symposium on Security and Privacy*. 47–62.

[13] Yinzhi Cao, Yanick Fratantonio, Antonio Bianchi, Antonio Chung, Yanick Huang, David Lie, Giovanni Vigna, and Christopher Kruegel. 2015. EdgeMiner: Automatically Detecting Implicit Control Flow Transitions through the Android Framework. In *Proceedings of the Network and Distributed System Security Symposium*.

[14] Haonan Chen, Daihang Chen, Yizhuo Yang, Lingyun Xu, Liang Gao, Mingyi Zhou, Chunming Hu, and Li Li. 2025. ArkAnalyzer: The Static Analysis Framework for OpenHarmony. In *Proceedings of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE SEIP)*. ACM.

[15] Haonan Chen, Mingyi Zhou, Yanjie Zhao, and Li Li. 2026. HapFlow: The Taint Analysis Framework for OpenHarmony Apps. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3793863

[16] Menglong Chen, Tian Tan, Minxue Pan, and Yue Li. 2025. PacDroid: A Pointer-Analysis-Centric Framework for Security Vulnerabilities in Android Apps. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*. 2803–2815. doi:10.1109/ICSE55347.2025.00208

[17] Yige Chen, Yujia Fan, Siyi Wang, Yi Tao, and Yepang Liu. 2025. HmTest: Automated Testing of HarmonyOS Apps via Model-Driven Navigation and Reinforcement Learning. *Journal of Computer Science and Technology* 40, 4 (2025), 1006–1021. doi:10.1007/s11390-025-5142-4

[18] Zhen Chen, Yi Li, Teng Wang, et al. 2025. HarmoBridge: Static Analysis of Cross-Language Data Flows in HarmonyOS Applications. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

[19] Erika Chin, Adrienne Porter Felt, Kate Greenwood, and David Wagner. 2011. Analyzing Inter-Application Communication in Android. In *Proceedings of the 9th International Conference on Mobile Systems, Applications, and Services*. 239–252.

[20] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2014. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. *ACM Trans. Comput. Syst.* 32, 2 (2014), 5:1–5:29.

[21] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. 2011. Android Permissions Demystified. In *Proceedings of the ACM Conference on Computer and Communications Security*. 627–638.

[22] Yu Feng, Saswat Anand, Isil Dillig, and Alex Aiken. 2014. Apposcopy: Semantics-Based Detection of Android Malware through Static Analysis. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 576–587.

[23] Stephen J. Fink, Julian Dolby, and Eran Yahav. 2008. Effective Typestate Verification in the Presence of Aliasing. *ACM Transactions on Software Engineering and Methodology* 17, 2, Article 9 (2008).

[24] Yanick Fratantonio, Antonio Bianchi, William Robertson, Dhilung Kirat, Christopher Kruegel, and Giovanni Vigna. 2016. TriggerScope: Towards Detecting Logic Bombs in Android Applications. In *Proceedings of the IEEE Symposium on Security and Privacy*. 377–396.

[25] Clint Gibler, Jonathan Crussell, Jeremy Erickson, and Hao Chen. 2012. AndroidLeaks: Automatically Detecting Potential Privacy Leaks in Android Applications on a Large Scale. In *Proceedings of the 5th International Conference on Trust and Trustworthy Computing*. 291–307.

[26] Michael I. Gordon, Deokhwan Kim, Jeff H. Perkins, Limei Gilham, Nguyen Nguyen, and Martin Rinard. 2015. Information Flow Analysis of Android Applications in DroidSafe. In *Proceedings of the Network and Distributed System Security Symposium*.

[27] Huawei. 2021. HarmonyOS 2 Security Technical White Paper. https://consumer.huawei.com/

[28] Huawei Developer. 2026. ArkTS: Dedicated Programming Language for the HarmonyOS Ecosystem. https://developer.huawei.com/consumer/en/arkts/

[29] William Klieber, Lori Flynn, Amar Bhosale, Limin Jia, and Lujo Bauer. 2014. Android Taint Flow Analysis for App Sets. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 1–6.

[30] Sungho Lee, Julian Dolby, and Sukyoung Ryu. 2016. HybriDroid: static analysis framework for Android hybrid applications. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Association for Computing Machinery, New York, NY, USA, 250–261. doi:10.1145/2970276.2970368
[31] Li Li, Alexandre Bartel, Tegawendé F. Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Octeau, and Patrick McDaniel. 2015. IccTA: Detecting Inter-Component Privacy Leaks in Android Apps. In *Proceedings of the 37th International Conference on Software Engineering*. 280–291.
[32] Li Li, Xiang Gao, Hailong Sun, Chunming Hu, Xiaoyu Sun, Haoyu Wang, Haipeng Cai, Ting Su, Xiapu Luo, Tegawendé F. Bissyandé, Jacques Klein, John Grundy, Tao Xie, Haibo Chen, and Huaimin Wang. 2025. Software Engineering for OpenHarmony: A Research Roadmap. *Comput. Surveys* 58, 2, Article 34 (2025), 34 pages. doi:10.1145/3720538
[33] Yi Li, Yanyan Chen, Teng Wang, et al. 2023. TaintMini: Detecting Privacy Data Leaks in Mini-Programs via Universal Data Flow Graph. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*.
[34] Zhihao Lin, Mingyi Zhou, Wei Ma, Chi Chen, Yun Yang, Jun Wang, Chunming Hu, and Li Li. 2025. HapRepair: Learn to Repair OpenHarmony Apps. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, Industry Track*. 319–330. doi:10.1145/3696630.3728556
[35] Farong Liu, Mingyi Zhou, Yakun Zhang, Ting Su, Bo Sun, Jacques Klein, Xiang Gao, and Li Li. 2025. HapTest: The Dynamic Analysis Framework for OpenHarmony. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, Companion Proceedings*. 1–10. doi:10.1145/3696630.3728565
[36] Long Lu, Zhichun Li, Zhenyu Wu, Wenke Lee, and Guofei Jiang. 2012. CHEX: Statically Vetting Android Apps for Component Hijacking Vulnerabilities. In *Proceedings of the ACM Conference on Computer and Communications Security*. 229–240.
[37] Linghui Luo, Felix Pauck, Goran Piskachev, Manuel Benz, Ivan Pashchenko, Martin Mory, Eric Bodden, Ben Hermann, and Fabio Massacci. 2022. TaintBench: Automatic Real-World Malware Benchmarking of Android Taint Analyses. *Empirical Software Engineering* 27, 1 (2022), 16. doi:10.1007/s10664-021-10013-5
[38] Damien Octeau, Somesh Jha, and Patrick McDaniel. 2013. Effective Inter-Component Communication Mapping in Android with Epicc: An Essential Step Towards Holistic Security Analysis. In *Proceedings of the 22nd USENIX Security Symposium*. 543–558.
[39] Damien Octeau, Daniel Luchaup, Matthew Dering, Somesh Jha, and Patrick McDaniel. 2015. Composite Constant Propagation: Application to Android Inter-Component Communication Analysis. In *Proceedings of the 37th International Conference on Software Engineering*. 77–88.
[40] Lucky Onwuzurike, Enrico Mariconti, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2019. MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models (Extended Version). *ACM Transactions on Privacy and Security (TOPS)* 22, 2, Article 10 (2019), 10:1–10:29 pages.
[41] OpenAtom Foundation. 2026. OpenHarmony: A Comprehensive Open Source Project for the All-scenario, Fully-connected, Intelligent Era. https://www.openharmony.cn/
[42] Felix Pauck, Eric Bodden, and Heike Wehrheim. 2018. Do Android Taint Analysis Tools Keep Their Promises?. In *Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 331–341. doi:10.1145/3236024.3236029
[43] Siegfried Rasthofer, Steven Arzt, and Eric Bodden. 2014. A Machine-learning Approach for Classifying and Categorizing Android Sources and Sinks. In *Proceedings of the Network and Distributed System Security Symposium*.
[44] Siegfried Rasthofer, Steven Arzt, Enrico Lovat, and Eric Bodden. 2016. HARVESTER: Automatic Generation of Entry Points for Dynamic Analysis of Android Applications. In *Proceedings of the Network and Distributed System Security Symposium*.
[45] Talia Ravitch, Eric Creswick, Aaron Tomb, Adam Foltzer, Trevor Elliott, and Ledah Casburn. 2014. Multi-App Security Analysis with Fuse: Statically Detecting Android App Collusion. In *Proceedings of the ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 4–9.
[46] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise Interprocedural Dataflow Analysis via Graph Reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 49–61.
[47] Mooly Sagiv, Thomas Reps, and Susan Horwitz. 1996. Precise Interprocedural Dataflow Analysis with Applications to Constant Propagation. *Theoretical Computer Science* 167, 1–2 (1996), 131–170.
[48] Jordan Samhi, Alexandre Bartel, Tegawendé F. Bissyandé, and Jacques Klein. 2021. RAICC: Revealing Atypical Inter-Component Communication in Android Apps. *Empirical Software Engineering* 26, 6 (2021).
[49] Kimberly Tam, Ali Feizollah, Nor Badrul Anuar, Rosli Salleh, and Lorenzo Cavallaro. 2015. CopperDroid: Automatic Reconstruction of Android Malware Behaviors. In *Proceedings of the Network and Distributed System Security Symposium*.
[50] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundaresan. 2010. Soot: A Java Bytecode Optimization Framework. In *CASCON First Decade High Impact Papers*. IBM Corp., 214–224.
[51] Yue Wang, Sen Chen, Jiaming Li, Wenjing Dang, Renchao Chai, Ziyue Shi, and Li Li. 2025. HarmonyFlow: A Static Analysis Framework for HarmonyOS Applications Based on Ark Panda IR. *Journal of Software* (2025), 1–19. doi:10.13328/j.cnki.jos.007477

[52] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. 2018. Amandroid: A Precise and General Inter-component Data Flow Analysis Framework for Security Vetting of Android Apps. *ACM Trans. Priv. Secur.* 21, 3 (2018), 14:1–14:32.

[53] Michelle Y. Wong and David Lie. 2016. IntelliDroid: A Targeted Input Generator for the Dynamic Analysis of Android Malware. In *Proceedings of the Network and Distributed System Security Symposium.*

[54] Haoyu Wu, Lingxiang Yang, Mengfei Wang, et al. 2023. WEMINT: Detecting Sensitive Data Flows in Mini-Programs via Explicit Asynchronous Callback Modeling. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE).*

[55] Meng Xia, Lu Gong, Yuan Lyu, Zhengwei Qi, and Xue Liu. 2015. Effective Real-time Android Application Auditing. In *Proceedings of the IEEE Symposium on Security and Privacy.* 899–914.

[56] Lok Kwong Yan and Heng Yin. 2012. DroidScope: seamlessly reconstructing the OS and Dalvik semantic views for dynamic Android malware analysis. In *Proceedings of the 21st USENIX Conference on Security Symposium (USENIX Security)* (Bellevue, WA). USENIX Association, USA, 29.

[57] Guangliang Yang, Abner Mendoza, and Guofei Gu. 2018. Automated Generation of Event-Oriented Exploits in Android Hybrid Apps. In *Proceedings of the Network and Distributed System Security Symposium.*

[58] Guangliang Yang, Abner Mendoza, Jialong Zhang, and Guofei Gu. 2017. BridgeScope: Precisely and Scalably Vetting JavaScript Bridge Security Issues in Android Hybrid Apps. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses.* 166–185.

[59] Wei Yang, Xusheng Xiao, Benjamin Andow, Sihan Li, Tao Xie, and William Enck. 2015. AppContext: Differentiating Malicious and Benign Mobile App Behaviors Using Context. In *Proceedings of the International Conference on Software Engineering.* 303–313.

[60] Yizhuo Yang, Lingyun Xu, Mingyi Zhou, and Li Li. 2026. Context-Sensitive Pointer Analysis for ArkTS. arXiv preprint. https://arxiv.org/abs/2602.00457

[61] Zhemin Yang and Min Yang. 2012. LeakMiner: Detect Information Leakage on Android with Static Taint Analysis. In *Proceedings of the 3rd World Congress on Software Engineering.* 101–104.

[62] Zhemin Yang, Min Yang, Yuan Zhang, Guofei Gu, Peng Ning, and X. Sean Wang. 2013. AppIntent: Analyzing Sensitive Data Transmission in Android for Privacy Leakage Detection. In *Proceedings of the ACM Conference on Computer and Communications Security.* 1043–1054.

[63] Le Yu, Xiapu Luo, Jiachi Chen, Hao Zhou, Tao Zhang, Henry Chang, and Hareton K. N. Leung. 2021. PPChecker: Towards Accessing the Trustworthiness of Android Apps' Privacy Policies. *IEEE Transactions on Software Engineering (TSE)* 47, 2 (2021), 221–242.

[64] Mu Zhang, Yue Duan, Heng Yin, and Zhiruo Zhao. 2014. Semantics-Aware Android Malware Classification Using Weighted Contextual API Dependency Graphs. In *Proceedings of the ACM Conference on Computer and Communications Security.* 1105–1116.

[65] Wu Zhou, Yajin Zhou, Xuxian Jiang, and Peng Ning. 2012. Detecting Repackaged Smartphone Applications in Third-party Android Marketplaces. In *Proceedings of the ACM Conference on Data and Application Security and Privacy.* 317–326.

[66] Yajin Zhou, Zhi Wang, Wu Zhou, and Xuxian Jiang. 2012. DroidRanger: Systematic Characterization and Detection of Malicious Applications in Official and Alternative Android Markets. In *Proceedings of the Network and Distributed System Security Symposium.*